FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

FH Aachen

Fachbereich Elektrotechnik und Informationstechnik

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc.)

Visualisierung und Generierung von SHACL-Regeln in einem UML Editor für CIM/XML Ontologien

Eingereicht von:

Maximilian Reisch

Matrikelnummer: 3525391

Studienrichtung: Informatik

08. August 2025

Betreuer: Prof. Dr. rer. nat. Andreas Claßen

Koreferent: Michael Lomb (B.Sc.), SOPTIM AG

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Aachen, im Monat Jahr (vollständige, handschriftliche Unterschrift)

Inhalt

1 Einleitung ... 1
2 Grundlagen ... 3
2.1 Resource Description Framework (RDF) ... 3
2.1.1 Blank Nodes ... 3
2.1.2 Serialisierung ... 4
2.1.3 SPARQL ... 4
2.1.4 RDF Schema (RDFS) ... 5
2.2 Unified Modeling Language (UML) ... 5
2.3 CIM/XML Ontologien ... 6
2.3.1 Ontologien ... 6
2.3.2 European Network of Transmission System Operators for Electricity (ENTSO-E) 6
2.3.3 Common Grid Model Exchange Standard (CGMES) ... 7
2.3.4 CIM-Modellierungskonzepte ... 7
2.4 RDF-Architect ... 9
2.5 Shape Constraint Language (SHACL) ... 10
2.5.1 Begrifflichkeiten und Hierarchie ... 11
2.5.2 Grundkonzepte und Architektur ... 11
2.5.3 Zentrale SHACL-Komponenten ... 11
2.6 Technologien ... 13
2.6.1 Backend ... 13
2.6.2 Frontend ... 14
2.6.3 REST ... 16
3 Anforderungen und Forschungsfragen ... 17
3.1 Anforderungskatalog ... 17
3.2 Forschungsfragen ... 18
4 Konzeption der SHACL-Erweiterung für den RDF-Architect UML-Editor ... 19
4.1 Analyse existierender SHACL-Dateien ... 19
4.1.1 owl:Ontology ... 20
4.1.2 sh:NodeShape ... 20
4.1.3 sh:PropertyShape ... 21
4.1.4 sh:PropertyGroups ... 24
4.1.5 Betrachtung weiterer SHACL-Dateien ... 24
4.2 SHACL-Generierung aus CIM/XML ... 25
4.2.1 CIM-Property zu PropertyShapes ... 25
4.2.2 CIM-Class zu NodeShape ... 32

4.2.3 PropertyGroups ... 32
4.3 Visualisierung von SHACL im RDF-Architect ... 33
4.3.1 Property-spezifische Ansicht ... 34
4.3.2 Klassenspezifische Ansicht ... 37
4.3.3 Vollständige Ansicht ... 38
4.3.4 Import-Dialog ... 39
4.3.5 Export Dialog ... 40
4.4 Frontend-Backend-Schnittstelle ... 41
5 Details zur Realisierung ... 42
5.1 Auswahl und Integration der Technologien ... 42
5.1.1 Backend Technologien ... 42
5.1.2 Frontend Technologien ... 42
5.2 Umsetzung der Generierung der SHACL-Regeln ... 43
5.2.1 Generierungsablauf ... 43
5.2.2 Implementierung des XSD Datatype-Mappings ... 47
5.2.3 Tests und Validierung der generierten SHACL-Regeln ... 47
5.3 Umsetzung der Visualisierung ... 48
5.3.1 Realisierung der Grafischen Darstellung ... 49
5.3.2 Backend ... 50
5.3.3 Import/Export ... 54
6 Fazit und Ausblick ... 56
6.1 Zielerreichung und Bewertung der Lösung ... 56
6.2 Limitationen der aktuellen Implementierung ... 57
6.3 Ausblick und Weiterentwicklungsmöglichkeiten ... 57
7 Quellenverzeichnis ... 58
8 Anhang ... 61
8.1 Queries ... 61
8.1.1 Auflistung und Quantifizierung der SHACL-Ressourcentypen ... 61
8.1.2 Quantifizierung und Zusammenhang von NodeShapes und Klassen ... 61
8.1.3 Relation von PropertyShapetypen zu Properties ... 62
8.2 Tests ... 63
8.2.1 Ontology.ttl ... 63
8.2.2 hasTypeWarningMissingReferencedClass.ttl ... 65

Listingverzeichnis

Listing 1: Beispiel Blank Nodes (Turtle Format) 3
Listing 2: Beispiel Blank Node rdf/xml Format 4
Listing 3: UML-Klassendiagramm 6
Listing 4: Beispiel CIM-Klasse 8
Listing 5: Beispiel CIM-Attribut 8
Listing 6: Beispiel CIM-Assoziation 9
Listing 7: RDF-Architect Hauptansicht 10
Listing 8: SHACL Beispiel 12
Listing 9: veröffentlichtes -datatype PropertyShape 22
Listing 10: Instanzbeispiel Assoziation 23
Listing 11: Muster -cardinality PropertyShape 26
Listing 12: Muster -datatype base-PropertyShape 27
Listing 13: Muster primitive -datatype PropertyShape 28
Listing 14: Muster enum -datatype PropertyShape 28
Listing 15: Muster -valueType PropertyShape 30
Listing 16: Muster -hasType PropertyShape 31
Listing 17: Muster -inverseCardinality PropertyShape 31
Listing 18: Muster NodeShape 32
Listing 19: Muster closed NodeShape Erweiterung 32
Listing 20: Muster PropertyGroup 32
Listing 21: class-Editor 34
Listing 22: Propertydarstellung im classeditor des RDF-Architect 35
Listing 23:Propertyspezifische Ansicht für generiertes SHACL 36
Listing 24: Propertyspezifische Ansicht für custom SHACL 36
Listing 25: Klassenspezifische Ansicht für generiertes SHACL 37
Listing 26: Klassenspezifische Ansicht für custom SHACL 38
Listing 27: Vollständige Ansicht für custom SHACL 39
Listing 28: Importdialog für custom SHACL 39
Listing 29: Importdialog für custom SHACL dropdown 40
Listing 30: Exportdialog für SHACL 40
Listing 31: Interfacemethoden PropertyShapeFromCIMPropertyGenerator 44
Listing 32: createPropertyShape Methode für inverseCardinality PropertyShapes 46
Listing 33: createNodeShape Methode 46
Listing 34: Test MissingReferencedRessource 48
Listing 35: TtlCodeEditor.svelte 49
Listing 36: Returnformat PropertyShapes spezifisch für Property 50
Listing 37: Query Zuweisung PropertyShape zu Property 51
Listing 38: Returnformat Klassenspezifische Ansicht 52
Listing 39: merge-Methode das Zusammenführen von custom und merged SHACL 54
Listing 40: copyStatementToModel-Methode für das korrekte Kopieren von Blank Nodes .. 55

1 Einleitung

Die Liberalisierung der Energiewirtschaft in Europa begann in den 1990er Jahren mit dem Ziel, Wettbewerb zu fördern, Preise zu senken und die Versorgungssicherheit zu verbessern. Durch die Entflechtung traditioneller Monopole entstanden neue Marktstrukturen und Akteure mit spezifischen Rollen wie Netzbetreiber, Energieversorger, Stromerzeuger und Regulierungsbehörden. Diese Umstrukturierung sollte Innovationen und Investitionen in erneuerbare Energien fördern und die Energiewende vorantreiben. [1]

Die European Network of Transmission System Operators for Electricity (ENTSO-E) ist der Verband der europäischen Übertragungsnetzbetreiber für Strom. Sie wurde gegründet, um die Zusammenarbeit zwischen den Netzbetreibern zu fördern und die Umsetzung der EU-Energiepolitik zu unterstützen. [2]

Der ENTSO-E Common Information Model (CIM) Standard spielt eine wichtige Rolle in der Liberalisierung der Energiewirtschaft, indem er eine einheitliche Datenstruktur für den Austausch zwischen verschiedenen Marktakteuren wie Übertragungsnetzbetreibern, Energieversorgern und deren Dienstleistern schafft. Der CIM-Standard wird für verschiedene Prozesse genutzt, darunter die Netzmodellierung für die Entwicklung neuer Stromnetze sowie die Marktkommunikation für Energiehandelsprozesse. Um die Modelle zwischen unterschiedlichen Software-Systemen auszutauschen, werden diese meist in ein maschinenlesbares XML-Format zu sogenannten CIM/XML-Ontologien serialisiert. [3]

Zur Validierung von Instanzdaten dieser CIM-Modelle werden strukturelle Validierungsregeln mittels der Shapes Constraint Language (SHACL) definiert. Diese SHACL-Regeln gewährleisten die Datenintegrität und sind somit ein wichtiger Teil der Sicherstellung der Fehler- und Störungsfreiheit sicherheitskritischer Prozesse in der Energieversorgung. [4]

Das Erstellen und das Aktualisieren dieser SHACL-Regeln sind momentan jedoch ein ineffizienter und fehleranfälliger Prozess. Es existieren derzeit keine Tools, die die gleichzeitige Bearbeitung des CIM-Modells und die Wartung der dazugehörigen SHACL-Regeln ermöglichen. Stattdessen müssen SHACL-Regeln nach Änderungen des CIM-Modells in einem eigenen Arbeitsschritt manuell nachgebessert werden, was zu Inkonsistenzen zwischen dem Datenmodell und den Validierungsregeln führen kann.

Die vorliegende Arbeit soll einen Lösungsansatz für die beschriebene Problematik bieten, indem das Erstellen von SHACL-Regeln nicht mehr als unabhängiger Arbeitsschritt gehandhabt wird, sondern deren Generierung und Wartung direkt in einen Editor für CIM-Modelle integriert werden. Dieser Ansatz eliminiert Inkonsistenzen und automatisiert die Synchronisation zwischen Modell und Validierungslogik.

Die Arbeit entsteht im Auftrag der SOPTIM AG, einem in der Energiewirtschaft seit über 50 Jahren etablierten Softwareunternehmen. [5] Dieses entwickelt seit Ende 2024 den "RDF-Architect", einen Editor für CIM/XML-Ontologien, welcher in der vorliegenden Arbeit um eine Implementierung des oben genannten Lösungsansatzes erweitert wird. Ziel der Arbeit ist die Entwicklung einer automatischen SHACL-Generierung mit benutzerfreundlicher Integration in den RDF-Architect.

Die Arbeit gliedert sich in acht Kapitel. Das erste Kapitel gibt einen ersten Einblick in die Thematik und beschreibt ein grobes Ziel. Kapitel 2 erläutert einige Grundlagen, die den fachlichen Kontext der Arbeit bilden bzw. als Technologien im Rahmen der Arbeit verwendet

werden. In Kapitel 3 wird ein detaillierter Anforderungskatalog darüber definiert, was die Implementierung bieten soll und welche Forschungsfragen sich daraus ergeben. Eine genaue Konzeption der im Rahmen der Arbeit entwickelten SHACL-Erweiterung erfolgt in Kapitel 4. Hier werden sowohl die Generierungslogik als auch eine benutzerfreundliche Visualisierung im RDF-Architect konzeptioniert. Kapitel 5 dokumentiert die technische Implementierung und Validierung der Lösung. Kapitel 6 fasst die Ergebnisse zusammen und gibt einen Ausblick auf weiterführende Entwicklungsmöglichkeiten. In Kapitel 7 werden die der Arbeit zu Grunde liegenden Quellen aufgeführt, während das 8. Kapitel die Anhänge beinhaltet.

2 Grundlagen

2.1 Resource Description Framework (RDF)

Das Resource Description Framework (RDF) ist ein vom World Wide Web Consortium (W3C) entwickelter Standard zur Beschreibung von Ressourcen im Web und bildet das Fundament für Linked Data und das Semantic Web [6]. Es wird zur Beschreibung von beliebigen Dingen (Ressourcen) genutzt und dient in dieser Arbeit als Datenmodell für CIM/XML-Ontologien und deren SHACL-Validierungsregeln. Diese Arbeit nutzt die Version 1.1 des RDF-Frameworks.

Jedes Statement in einem RDF-Modell wird über ein Triple getätigt. Diese Triples bestehen aus jeweils einem Subjekt, einem Prädikat und einem Objekt. Ein Triple könnte beispielweise etwas aussagen wie: Die Person (Subjekt) hat den Namen (Prädikat) Max Mustermann (Objekt). In welchem Format die einzelnen Teile des Triples sein dürfen, ist eingeschränkt [7].

Ein Subjekt darf eine IRI (Internationalized Ressource Identifier) oder eine Blank Node sein [7]. IRIs sind eindeutige Identifikatoren einer Ressource und stellen somit sicher, dass es keine Namenskonflikte zwischen diesen gibt [8] [9]. Um die Lesbarkeit zu verbessern, werden lange IRIs durch Präfix-Notation verkürzt. Dabei wird der gemeinsame Namespace-Teil einer IRI durch ein kurzes Präfix ersetzt. Die IRI `http://example.com#name` wird beispielsweise zu `ex:name`, wobei das Präfix `ex:` für `http://example.com#` steht. Es ist außerdem möglich, ein Basispräfix einzustellen, welches für einen singulären Doppelpunkt `:` substituiert wird. [10]

Ein Prädikat muss eine IRI sein und ein Objekt darf entweder eine IRI, eine Blank Node oder ein typisiertes Literal sein [7] [11]. Ein typisiertes Literal ist einfach ein Freitext, welchem ein beliebiger Datentyp zugewiesen wird. Um semantisch korrekt zu sein, könnte das o.g. genannte Beispiel folgendermaßen aussehen: `http://example.com#Person http://example.com#name "Max Mustermann"^^xmlLiteral.`

2.1.1 Blank Nodes

Eine Blank Node ist eine Anonyme Ressource. Das bedeutet, dass sie keine IRI besitzt. Diese werden beispielsweise genutzt, um komplexere Eigenschaften einer Ressource zu beschreiben [12]. Die beiden in Listing 1 dargestellten Ressourcen beschreiben den Namen einer Person, wobei für die Ressource für "Marie Musterfrau" eine Blank Node verwendet wird, um den Vor- und Nachnamen zu bündeln.

```
@prefix ex: <http://example.com#>.
@prefix xml: <http://www.w3.org/2001/XMLSchema#>.
ex:Person1
    ex:vorname  "Max"^^xml:Literal;
    ex:vorname  "Mustermann"^^xml:Literal.

ex:Person2
    ex:name     _:b1.

_:b1
    ex:vorname  "Marie"^^xml:Literal;
    ex:nachname "Musterfrau"^^xml:Literal.
```

Listing 1: Beispiel Blank Nodes (Turtle Format)

Eine Menge dieser Triples bildet einen gerichteten Graphen und wird auch als RDF-Modell bezeichnet. Dabei fungieren Ressourcen (Subjekte und Objekte) als Knoten und Prädikate als gerichtete Kanten zwischen diesen Knoten. [13]

2.1.2 Serialisierung

RDF kann in verschiedene Formate serialisiert werden. In dieser Arbeit wird das auch in Listing 1 verwendete Turtle (ttl) Format genutzt, da es relativ einfach lesbar ist [14]. Es existieren jedoch auch andere Formate wie zum Beispiel das in Listing 2 gezeigte und in der Energiewirtschaft gängige RDF/XML-Format. [15]

```
<?xml version="1.0" encoding="utf-8" ?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:ns0="http://example.com#">
  <rdf:Description rdf:about="http://example.com#Person1">
    <ns0:vorname rdf:datatype="http://www.w3.org/2001/XMLSchema#Literal">
        Max
    </ns0:vorname>
    <ns0:vorname rdf:datatype="http://www.w3.org/2001/XMLSchema#Literal">
        Mustermann
    </ns0:vorname>
  </rdf:Description>
  <rdf:Description rdf:about="http://example.com#Person2">
    <ns0:name>
      <rdf:Description>
        <ns0:vorname rdf:datatype="http://www.w3.org/2001/XMLSchema#Literal">
            Marie
        </ns0:vorname>
        <ns0:nachname rdf:datatype="http://www.w3.org/2001/XMLSchema#Literal">
            Musterfrau
        </ns0:nachname>
      </rdf:Description>
    </ns0:name>
  </rdf:Description>
</rdf:RDF>
```

Listing 2: Beispiel Blank Node rdf/xml Format

Neben den beiden genannten Formaten gibt es noch weitere wie unter anderem JSON-LD, N-Triples und weitere, welche jedoch für diese Arbeit nicht weiter relevant sind.

2.1.3 SPARQL

Um diese Graphen systematisch zu durchsuchen, wird die SPARQL Protocol and RDF Query Language (SPARQL) eingesetzt. Diese W3C-Standardsprache ermöglicht strukturierte Abfragen von RDF-Daten mittels deklarativer Query-Syntax. SPARQL funktioniert nach dem Prinzip des Graph-Pattern-Matching: Eine Abfrage definiert ein Muster aus Subjekt-Prädikat-Objekt-Strukturen, das gegen den RDF-Graphen geprüft wird. Dabei können Variablen, welche durch ein "?" gekennzeichnet werden, als Platzhalter für unbekannte Werte dienen. Die grundlegende Syntax folgt dem SELECT-FROM-WHERE-Schema von SQL, wobei komplexere Abfragen durch UNION-Operationen, FILTER-Bedingungen und verschachtelte Muster

realisiert werden können. In dieser Arbeit wird SPARQL vor allem zur Analyse der Ontologie-Struktur und zur programmatischen Zuordnung von SHACL-PropertyShapes zu CIM-Properties eingesetzt. [16]

2.1.4 RDF Schema (RDFS)

Das RDF-Framework wird durch RDF Schema (RDFS) um grundlegende Modellierungskonzepte erweitert, die für die Strukturierung von RDF-Daten erforderlich sind. RDFS definiert Klassen- und Eigenschaftshierarchien sowie deren Beziehungen. Zugehörige IRIs sind im Namespace `rdfs:` bzw. `http://www.w3.org/2000/01/rdf-schema#`. Zentrale RDFS-Prädikate wie `rdfs:Class` kennzeichnen Ressourcen als Klassen, `rdfs:subClassOf` etabliert Vererbungsbeziehungen, `rdfs:domain` und `rdfs:range` bestimmen Definitionsbereich und Wertebereich von Properties, und `rdfs:label` stellt menschenlesbare Bezeichnungen bereit. [17]

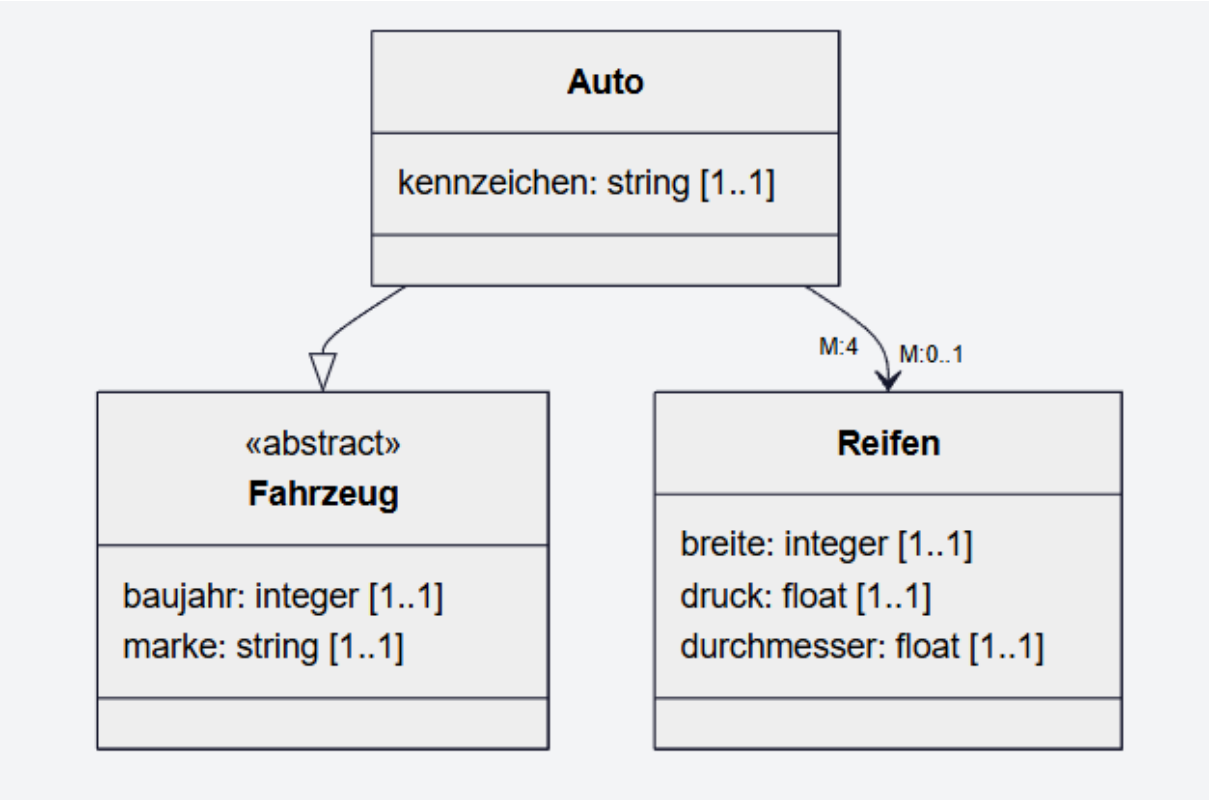
Der Standard-RDF-Namespace `rdf:` mit der IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#` umfasst grundlegende Prädikate wie `rdf:type` zur Typisierung von Ressourcen oder `rdf:Property` zur Kennzeichnung von Properties. Diese RDFS- und RDF-Vokabulare bilden die strukturelle Grundlage für die in dieser Arbeit verwendeten CIM/XML-Ontologien. [17]

2.2 Unified Modeling Language (UML)

Die Unified Modeling Language (UML) ist eine Modellierungssprache zur graphischen Darstellung von Software- und anderen Systemteilen. Sie bietet verschiedene Diagrammtypen für verschiedene Anwendungsbereiche. [18] Im Kontext dieser Arbeit sind jedoch besonders die UML-Klassendiagramme relevant.

Klassendiagramme werden genutzt, um die statische Struktur eines Systems zu beschreiben. In dieser Anwendung werden vor allem die Grundelemente Klassen, Attribute, Assoziationen, Vererbungen und Multiplizitäten genutzt. [19]

Das in Listing 3 dargestellte Klassendiagramm beschreibt beispielsweise, dass eine Klasse "Auto" Eigenschaften der Klasse "Fahrzeug" erbt. Sie hat dadurch neben dem Attribut "kennzeichen" zusätzlich noch die Attribute "baujahr" und "marke". Sie verweist außerdem über eine Assoziation auf die Klasse "Reifen". Die Multiplizitätsangaben sagen aus, dass ein Auto vier Reifen haben muss, ein Reifen aber nur maximal an einem Auto sein kann. Jeder Reifen hat außerdem auch wiederum mehrere Attribute. Darüber hinaus nutzt UML Stereotypen zur Kennzeichnung spezieller Klasseneigenschaften. Die abstrakte Klasse "Fahrzeug" ist durch den Stereotyp <<abstract>> gekennzeichnet und kann nicht direkt instanziiert werden. Nur für konkrete Unterklassen wie "Auto" können Objektinstanzen erstellt werden.



Listing 3: UML-Klassendiagramm

Diese UML-Konzepte werden im Common Information Model (CIM) Standard konkret für die Energiewirtschaft angewendet, um einheitliche Datenstrukturen für den Austausch zwischen verschiedenen Systemen zu schaffen.

2.3 CIM/XML Ontologien

2.3.1 Ontologien

Eine Ontologie ist ein formales Modell zur Beschreibung und Strukturierung von Wissen in einem bestimmten Bereich. Sie definiert Begriffe, deren Beziehungen zueinander und die Regeln für deren Verwendung. Im Kontext der Informatik ermöglichen Ontologien die maschinenlesbare Repräsentation von Domänenwissen, wodurch Software-Anwendungen semantische Zusammenhänge verstehen und verarbeiten können. [20]

2.3.2 European Network of Transmission System Operators for Electricity (ENTSO-E)

Die ENTSO-E ist der Verband der europäischen Übertragungsnetzbetreiber für Strom. Eine zentrale Aufgabe von ENTSO-E besteht in der Standardisierung des Datenaustauschs zwischen den verschiedenen Akteuren der liberalisierten Energiewirtschaft. [2]

Der Common Information Model (CIM) Standard wird von der International Electrotechnical Commission (IEC) verwaltet und von der ENTSO-E für europäische Anwendungen erweitert und angepasst. [3] Bei der Verwendung des Begriffs "Common Information Model" ist eine wichtige Abgrenzung zu beachten: Der in dieser Arbeit behandelte IEC CIM Standard unterscheidet sich grundlegend vom Common Information Model (CIM) der Distributed

Management Task Force (DMTF). Während das DMTF CIM als Standard zur Verwaltung von IT-Systemen, Netzwerken und Anwendungen dient, wurde der IEC CIM primär für die Modellierung von Energiesystemen entwickelt. [21] [22]

CIM-Ontologien beschreiben die Komponenten und Beziehungen von Energiesystemen in standardisierter Form unter Verwendung von UML-Klassendiagramm-Strukturen: Kraftwerke, Übertragungsleitungen, Transformatoren, Schaltanlagen und deren elektrische sowie topologische Verbindungen werden als Klassen mit Attributen und Assoziationen modelliert. [22]

Um die Portabilität zwischen verschiedenen IT-Systemen zu gewährleisten, werden diese CIM-Ontologien in maschinenlesbare XML-Dokumente serialisiert, die als CIM/XML-Ontologien bezeichnet werden. Diese bilden die technische Grundlage für den automatisierten Datenaustausch in der europäischen Energieinfrastruktur. [3]

2.3.3 Common Grid Model Exchange Standard (CGMES)

Der Common Grid Model Exchange Standard (CGMES) ist eine spezialisierte Anwendung des IEC CIM-Standards, die von ENTSO-E entwickelt wurde, um den spezifischen Anforderungen des europäischen Stromübertragungsnetzes gerecht zu werden. CGMES basiert auf mehreren IEC CIM Standards und erweitert diese um energiewirtschaftsspezifische Funktionalitäten. [23]

Während CIM das generische Framework zur Modellierung von Energiesystemen bereitstellt, definiert CGMES konkrete Austauschprofile für den praktischen Datenaustausch zwischen Übertragungsnetzbetreibern (Transfer System Operators, TSOs). Diese Profile strukturieren die CIM-Ontologien in spezifische Anwendungsbereiche wie unter anderem Equipment-Beschreibungen für physische Netzkomponenten, Diagramm Layout-Profile für graphische Netzdarstellungen und State Variables für Lastflussberechnungen. [23]

In der Praxis werden hauptsächlich zwei CGMES-Versionen verwendet: CGMES 2.4.15, das als bewährter Standard für operative Anwendungen etabliert ist, und CGMES 3.0, welches als neuere Version erweiterte Funktionalitäten und verbesserte Interoperabilität bietet. CGMES ist seit 2021 als internationaler IEC-Standard etabliert und bildet die technische Grundlage für die europaweite Netzmodellierung und -koordination.

2.3.4 CIM-Modellierungskonzepte

CIM nutzt RDF-basierte Strukturen zur Darstellung von UML-Konzepten und erweitert diese um spezifische Stereotypen und Konventionen. Diese Stereotypen definieren die exakte semantische Bedeutung und steuern die Instanziierbarkeit von Klassen in der RDF-Repräsentation.

Das Präfix `cims:` steht für "CIM Schema" und referenziert den Namespace `http://iec.ch/TC57/1999/rdf-schema-extensions-19990926#`. Dieser IEC-Standard erweitert das Standard-RDF-Schema um CIM-spezifische Eigenschaften und Metadaten, die für die präzise Modellierung von Energiesystemen erforderlich sind. CIMS-Prädikate wie `cims:stereotype`, `cims:multiplicity` oder `cims:dataType` ergänzen die Standard-RDF-Vokabulare um domänenspezifische Semantik.

Um die CIM-Modellierung zu veranschaulichen, werden im Folgenden Teile des in Listing 3 gezeigten UML-Beispiels mit Fahrzeug, Auto und Reifen in CIM-Notation dargestellt.

CIM-Klassen

CIM-Klassen werden wie in Listing 4 sichtbar durch das Triple `rdf:type rdfs:Class` als Klassen identifiziert. Das `rdfs:label` Prädikat stellt eine menschenlesbare Bezeichnung bereit, während `cims:stereotype` die semantische Bedeutung und Instanziierbarkeit definiert. Vererbungsbeziehungen werden über `rdfs:subClassOf` etabliert.

```
cim:Auto
    rdf:type                rdfs:Class ;
    rdfs:label              "Auto"@en ;
    rdfs:subClassOf         cim:Fahrzeug ;
    cims:stereotype         <http://iec.ch/TC57/NonStandard/UML#concrete> .
```

Listing 4: Beispiel CIM-Klasse

Der Stereotyp `http://iec.ch/TC57/NonStandard/UML#concrete` kennzeichnet die Auto-Klasse als instanziierbar, während andere Stereotypen wie `enumeration` oder `description` unterschiedliche semantische Bedeutungen haben und die Instanziierbarkeit beeinflussen.

CIM-Attribute

CIM-Attribute wie das Beispiel in Listing 5 werden als RDF-Properties modelliert und durch den Stereotyp `http://iec.ch/TC57/NonStandard/UML#attribute` als solche gekennzeichnet. Das `rdfs:domain` Prädikat ordnet das Attribut einer Klasse zu. `cims:multiplicity` definiert die Kardinalität. Falls das Attribut einen primitiven Datentypen besitzt wird dieser durch `cims:dataType` referenziert. Falls es sich jedoch um einen nicht primitiven-Datentypen handelt geschieht dies durch `rdfs:range`.

```
cim:Auto.kennzeichen
    rdf:type                rdf:Property ;
    rdfs:label              "kennzeichen"@en ;
    rdfs:domain             cim:Auto ;
    cims:dataType           cim:String ;
    cims:stereotype         <http://iec.ch/TC57/NonStandard/UML#attribute> ;
    cims:multiplicity       cims:M:1..1 .
```

Listing 5: Beispiel CIM-Attribut

Die Namenskonvention `Klassenname.Attributname` gewährleistet eindeutige Identifikation, auch wenn mehrere Klassen gleichnamige Attribute besitzen.

CIM-Assoziationen

CIM-Assoziationen wie das Beispiel in Listing 6 werden als zwei separate RDF-Properties modelliert. Das `rdfs:range` Prädikat definiert die Zielklasse, `cims:AssociationUsed` bestimmt die primäre Nutzungsrichtung und `cims:inverseRoleName` etabliert die bidirektionale Verbindung. Das Statement `cims:AssociationUsed "Yes"` kennzeichnet die eigentliche Navigationsrichtung der Assoziation, wobei diese sowohl einseitig als auch bidirektional sein darf.

```
cim:Auto.Reifen
    rdf:type                rdf:Property ;
    rdfs:label              "Reifen"@en ;
    rdfs:domain             cim:Auto ;
    rdfs:range              cim:Reifen ;
    cims:AssociationUsed    "Yes" ;
    cims:inverseRoleName    cim:Reifen.Auto ;
    cims:multiplicity       cims:M:4..4 .

cim:Reifen.Auto
    rdf:type                rdf:Property ;
    rdfs:label              "Auto"@en ;
    rdfs:domain             cim:Reifen ;
    rdfs:range              cim:Auto ;
    cims:AssociationUsed    "No" ;
    cims:inverseRoleName    cim:Auto.Reifen ;
    cims:multiplicity       cims:M:0..1 .
```

Listing 6: Beispiel CIM-Assoziation

Kardinalitäts-Notation

Die CIM-Kardinalität wird über das `cims:multiplicity` Prädikat im Format `cims:M:n..m` angegeben, wobei `n` die Mindestanzahl und `m` die Höchstanzahl darstellt. Das Präfix `M:` kennzeichnet dabei eine Multiplizitätsangabe. Bei identischen Werten wie `cims:M:4..4` ist exakt diese Anzahl erforderlich, während `cims:M:0..1` optional (null oder eins) und `cims:M:1..*` mindestens eins mit unbegrenzter Obergrenze bedeutet. Es gibt außerdem alternative verkürzte Schreibweisen. Falls Unter- und Obergrenze gleich sind wird nur `cims:M:4` geschrieben und falls die Untergrenze null ist, wird nur die Obergrenze als `cims:M:m` angegeben wird.

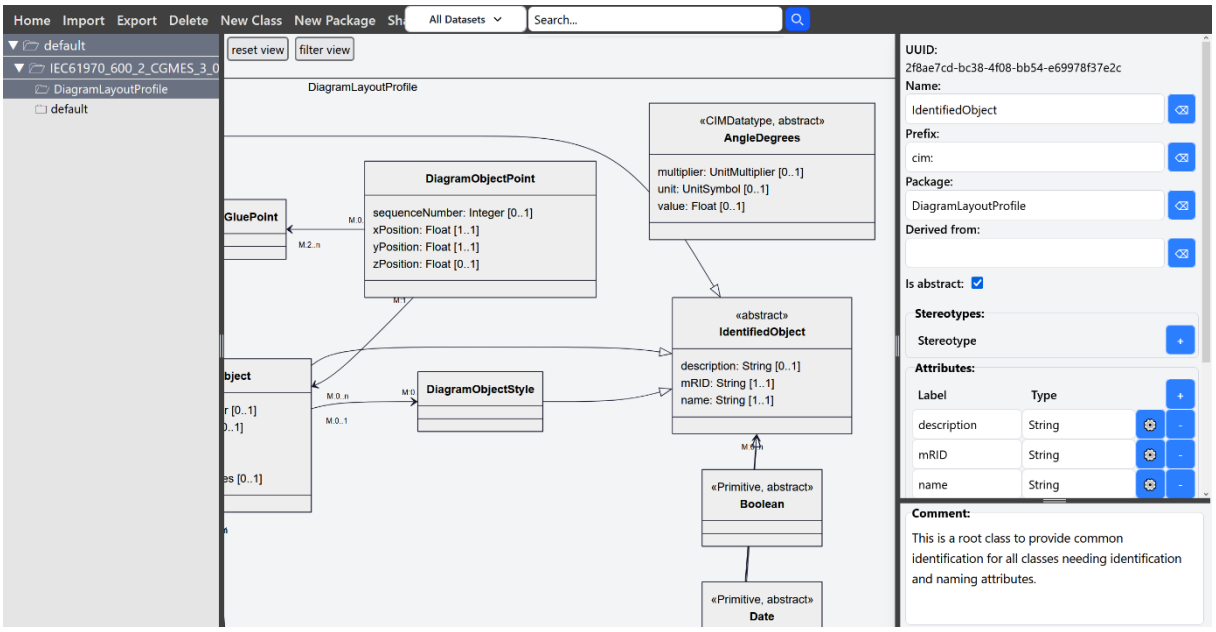
2.4 RDF-Architect

Der RDF-Architect ist eine von der SOPTIM AG entwickelte Webanwendung zur Modellierung und Verwaltung von CIM/XML-Ontologien. Das Tool wurde als moderne Alternative zum Enterprise Architect von Sparx Systems konzipiert, um die spezifischen Anforderungen der Energiewirtschaft bei der Pflege von CGMES-Datenmodellen zu erfüllen.

Die Entwicklung des RDF-Architects resultiert aus konkreten Problemen des bisherigen Workflows. Teams der SOPTIM nutzen RDF-Schemata basierend auf dem CGMES-Standard der ENTSO-E für Code- und Modellgenerierung. Die bisherige Pflege erfolgte mit Enterprise

Architect in Kombination mit speziellen Plug-ins, was jedoch zu einem fragilen System mit regelmäßiger Datenkorruption und fehlenden Funktionalitäten führte.

Der RDF-Architect visualisiert CIM/XML-Ontologien als UML-Klassendiagramme und bietet Funktionalitäten, diese zu bearbeiten (siehe Listing 7). Das Tool fungiert somit im Wesentlichen als spezialisierter UML-Editor, dessen Datenformat auf dem CIM-Standard basiert. Der RDF-Architect wird primär für die Kompatibilität mit CGMES 3.0.2 entwickelt, ist jedoch weitreichend auch mit CGMES 2.4.15 kompatibel.



Listing 7: RDF-Architect Hauptansicht

Das System folgt einer modernen Web-Architektur mit klarer Frontend-Backend-Trennung. Das Backend basiert auf Java mit dem Spring Framework und nutzt Apache Jena für die RDF-Verwaltung. Die Kommunikation erfolgt über eine RESTful API. Das Frontend nutzt JavaScript mit SvelteKit und Svelte 5, wobei mermaid.js für das Rendering der UML-Graphen verantwortlich ist.

Der RDF-Architect bildet die technische Grundlage für die in dieser Arbeit entwickelte SHACL-Erweiterung, welche die automatische Generierung und Visualisierung von Validierungsregeln für CIM/XML-Ontologien ermöglicht.

2.5 Shape Constraint Language (SHACL)

Die Shape Constraint Language (SHACL) ist ein vom World Wide Web Consortium (W3C) entwickelter Standard zur Validierung von RDF-Daten. Als deklarative Constraint-Sprache fungiert SHACL als Validierungswerkzeug für RDF-Daten und ermöglicht, strukturelle und inhaltliche Konsistenz von RDF-Graphen sicherzustellen. [24]

Für CIM/XML-Ontologien ist SHACL besonders relevant, da diese in RDF gespeichert werden und einer strukturellen Validierung bedürfen, um die Datenintegrität zu gewährleisten. Die ENTSO-E stellt zu CGMES 3.0.2 entsprechende SHACL-Regelsätze zur Verfügung, die in einer

strukturierten Verzeichnishierarchie organisiert sind. Für andere CIM-basierte Datensätze wie beispielsweise dem CGMES 2.4.15 gibt es jedoch gar keine SHACL-Regeln.

2.5.1 Begrifflichkeiten und Hierarchie

Um Verwirrungen zu vermeiden, werden die SHACL-Begrifflichkeiten in dieser Arbeit klar definiert und einheitlich verwendet. Der Begriff "SHACL-Regeln" bezeichnet das gesamte Konstrukt mitsamt allen Shapes für eine Ontologie. Innerhalb dieser SHACL-Regeln werden "Shapes" als einzelne Validierungseinheiten verstanden, die sich auf konkrete Ressourcen beziehen. Dabei wird zwischen NodeShapes für ganze Ressourcen und PropertyShapes für spezifische Eigenschaften unterschieden. Die kleinste Einheit bilden die Constraints, welche die einzelnen validierenden Eigenschaften innerhalb von Shapes darstellen, beispielsweise `sh:path` oder `sh:minCount`. [25]

Zusätzlich existieren weitere Constraint-Typen wie die Class-Constraints `sh:class` oder Pattern-Constraints durch `sh:pattern` für die Regex-Validierung. Durch die SPARQL-Constraints `sh:sparql` bietet SHACL zusätzlich nahezu unbegrenzte Validierungsmöglichkeiten für komplexe domänenspezifische Anforderungen. [26]

2.5.2 Grundkonzepte und Architektur

Bei der SHACL-Validierung werden zwei Graphen benötigt: Der Shapes Graph und der Data Graph. Der Shapes Graph enthält die Validierungsregeln, während der Data Graph die zu validierenden Instanzdaten beinhaltet. [27]

Der SHACL-Validierungsprozess erzeugt einen strukturierten Validation Report, der detaillierte Informationen über Validierungsergebnisse enthält. Dabei werden Verletzungen nach Schweregrad kategorisiert: `sh:Violation` für kritische Fehler, `sh:Warning` für Warnungen und `sh:Info` für informative Hinweise. [28]

SHACL-Regeln sind in unserem Kontext ontologiespezifisch. Das bedeutet, dass ein Regelsatz in Relation zu einer Ontologie steht und überprüft, ob die Instanzdaten die spezifischen, durch diese Ontologie definierten Zusammenhänge erfüllen.

2.5.3 Zentrale SHACL-Komponenten

NodeShapes (`sh:NodeShape`) definieren Validierungsregeln für ganze Ressourcen oder Knoten im RDF-Graph. Sie spezifizieren über Target-Definitionen wie z.B. `sh:targetClass`, welche Ressourcen validiert werden sollen und referenzieren über das `sh:property` Prädikat die PropertyShapes, welche auf die Ressource angewendet werden sollen. [29]

PropertyShapes (`sh:PropertyShape`) validieren spezifische Eigenschaften von Ressourcen. Die Definition darüber, welche Eigenschaft der Ressource überprüft werden soll, wird über `sh:path` definiert und kann sowohl einfache Eigenschaftspfade als auch komplexe Pfadausdrücke umfassen. [29]

PropertyGroups (`sh:PropertyGroup`) ermöglichen die organisatorische Gruppierung verwandter PropertyShapes. Sie verbessern die Übersichtlichkeit und Wartbarkeit komplexer

SHACL-Regelsätze, indem sie thematisch zusammengehörige Validierungsregeln strukturieren.

SHACL bietet verschiedene Constraint-Typen für die in dieser Arbeit relevanten Validierungsszenarien. Kardinalitäts-Constraints wie `sh:minCount` und `sh:maxCount` überprüfen die Anzahl von Eigenschaftswerten einer Ressource und stellen sicher, dass beispielsweise `required` Properties vorhanden sind oder die maximale Anzahl von Assoziationen nicht überschritten wird. Datentyp-Constraints überprüfen mittels `sh:datatype` den Datentyp von Literalwerten, während Wertebereichs-Constraints durch `sh:in` auf erlaubte Werte in einer vordefinierten Liste beschränken. Diese Darstellung eines exemplarischen Ausschnittes aus den existierenden Contraints soll einen Einblick über deren grundsätzliche Funktionsweise geben. [26]

Die in Listing 8 beschriebenen SHACL-Regeln validieren einen unvollständigen Ausschnitt aus dem UML Beispiel in Listing 3, Listing 4, Listing 5 und Listing 6. Hier wird beispielhaft illustriert, wie SHACL-Validierung funktioniert. Es wird validiert, dass eine Instanz der Klasse `cim:Auto` mittels einer Assoziation `cim:Auto.Reifen` auf 4 Reifen verweist.

```
# NodeShape für Auto-Klasse
ex:Auto
    rdf:type          sh:NodeShape ;
    sh:targetClass    cim:Auto ;
    sh:property       ex:Auto.Reifen-cardinality .

# PropertyShape für Reifen-Kardinalität
ex:Auto.Reifen-cardinality
    rdf:type          sh:PropertyShape ;
    sh:path           cim:Auto.Reifen ;
    sh:minCount       4 ;
    sh:maxCount       4 .

# Datagraph/Instanzen
# Auto-Instanz - valide
:meinAuto
    rdf:type          cim:Auto ;
    cim:Auto.Reifen :reifen1, :reifen2, :reifen3, :reifen4 .

# Auto-Instanz – invalide (nur 3 reifen)
:defektesAuto
    rdf:type          cim:Auto ;
    cim:Auto.Reifen :reifen1, :reifen2, :reifen3 .
```

Listing 8: SHACL Beispiel

Darüber, dass das NodeShape `ex:Auto` mittels `sh:targetClass` auf `cim:Auto` zeigt, wird die über `sh:property` referenzierte PropertyShape `ex:Auto.Reifen-cardinality` auf alle Instanzen mit dem `rdf:type cim:Auto` angewendet. Hier ist die `:meinAuto` Instanz valide, weil der in der PropertyShape definierte Path `cim:Auto.Reifen` mindestens und maximal viermal vorkommt. Bei der Instanz `:defektesAuto` schlägt die Validierung fehl, da die Relation nur dreimal existiert.

Diese grundlegenden SHACL-Konzepte bilden die Basis für die automatische Generierung von Validierungsregeln aus CIM/XML-Ontologien wie sie in den folgenden Kapiteln entwickelt wird.

2.6 Technologien

Die folgenden Abschnitte beschreiben die eingesetzten Technologien und deren jeweilige Rolle in der Gesamtarchitektur. Die Technologiewahl orientierte sich vor allem an den bereits im RDF-Architect genutzten Technologien sowie an bewährten Standards der Enterprise-Softwareentwicklung. Dabei lag der Fokus auf der nahtlosen Integration in die bestehende Systemlandschaft und der Minimierung zusätzlicher Abhängigkeiten. Zusätzlich müssen alle verwendeten Technologien mit der Apache License 2.0 kompatibel sein, um lizenzrechtliche Konflikte zu vermeiden.

2.6.1 Backend

Das Backend basiert auf etablierten Java-Technologien, die eine robuste und skalierbare Serverarchitektur ermöglichen. Die gewählten Frameworks und Bibliotheken bieten dabei umfassende Unterstützung für Enterprise-Anwendungen und RDF-Datenverarbeitung.

Java Spring

Das von Pivotal (VMware) entwickelte Spring Framework [30, 31] vereinfacht die Java-Entwicklung durch Dependency Injection, wodurch Abhängigkeiten zwischen Komponenten automatisch verwaltet werden. Dies reduziert Boilerplate-Code erheblich und macht die Anwendung testbarer. Spring Boot [32, 33] erweitert dies durch automatische Konfiguration. Anstatt manueller XML-Konfiguration erkennt Spring Boot die benötigten Abhängigkeiten selbst und konfiguriert diese automatisch.

Für REST-APIs bietet Spring Boot mit wenigen Annotationen wie `@RestController` und `@RequestMapping` eine einfache Möglichkeit, HTTP-Endpunkte zu erstellen. Das Framework übernimmt dabei automatisch die JSON-Serialisierung, Fehlerbehandlung und HTTP-Status-Codes. [34] Diese Eigenschaften machen Spring Boot besonders geeignet für die Backend-Entwicklung moderner Web-Anwendungen mit klarer Frontend-Backend-Trennung.

Apache Jena RDF API

Apache Jena [35, 36] ist eine von der Apache Software Foundation entwickelte Java-Bibliothek für die Arbeit mit RDF-Daten und bietet umfassende APIs für das Lesen, Schreiben und Verarbeiten von RDF-Graphen. Die Bibliothek unterstützt alle gängigen RDF-Serialisierungsformate wie Turtle, RDF/XML und JSON-LD, wodurch eine flexible Datenintegration ermöglicht wird. [37]

Das zentrale Konzept in Jena ist die `Model` Klasse, welche eine Sammlung von RDF-Triples repräsentiert. Ein Model fungiert als Container für RDF-Statements und bietet Methoden zum Hinzufügen, Entfernen und Abfragen von Triples. Durch diese Abstraktion können RDF-

Graphen als Java-Objekte manipuliert werden, ohne sich um die zugrundeliegende Serialisierung kümmern zu müssen. [37]

Ein zentraler Vorteil von Jena ist die integrierte SPARQL-Engine, die es ermöglicht, komplexe Abfragen direkt auf RDF-Graphen auszuführen, ohne externe Datenbanken zu benötigen. [38] Zusätzlich bietet Jena ein SHACL-Modul für die Validierung von RDF-Daten gegen Constraint-Regeln. [39] Diese Kombination aus RDF-Verarbeitung, SPARQL-Queries und SHACL-Validierung macht Jena zur idealen Wahl für RDF-basierte Anwendungen.

JUnit 5 und Assertj

Für die Validierung der SHACL-Generierungslogik werden JUnit 5 [40, 41] und AssertJ [42, 43] als Testing-Framework eingesetzt. JUnit 5 bietet moderne Testing-Features wie parametrisierte Tests [44], die es ermöglichen, die generierten SHACL-Regeln systematisch gegen verschiedene Testdaten und Szenarien zu prüfen.

AssertJ ergänzt JUnit durch eine Fluent-API, die detaillierte Assertions für SHACL-spezifische Validierungen ermöglicht. Die Bibliothek bietet aussagekräftige Fehlermeldungen bei fehlgeschlagenen Tests und eine intuitive Syntax für die Überprüfung komplexer RDF-Strukturen. [45] Da beide Technologien bereits im RDF-Architect etabliert waren, konnten bestehende Test-Patterns und -Konventionen wie das Arrange-Act-Assert Pattern übernommen werden, was die Konsistenz der Teststruktur gewährleistet.

Lombok

Lombok [46, 47] reduziert Boilerplate-Code durch automatische Generierung von Standard-Java-Methoden zur Compile-Zeit. Die `@Data`-Annotation generiert automatisch Getter, Setter, `equals()`, `hashCode()` und `toString()`-Methoden [48], während `@Builder` eine fluent API für die Objektkonstruktion bereitstellt [49].

In der SHACL-Generierung werden zahlreiche Datenklassen für CIM-Strukturen und SHACL-Shapes verwendet. Lombok eliminiert die manuelle Implementierung dieser Standard-Methoden und vereinfacht die Konstruktion komplexer SHACL-Objekte erheblich. Da die Bibliothek bereits im RDF-Architect etabliert war, konnte diese Code-Vereinfachung nahtlos übernommen werden.

2.6.2 Frontend

Das Frontend nutzt moderne JavaScript-Technologien für eine reaktive und benutzerfreundliche Oberfläche. Die gewählten Frameworks und Bibliotheken ermöglichen eine komponentenbasierte Entwicklung mit effizientem Code-Highlighting und -Bearbeitung.

Svelte-Kit

SvelteKit [50, 51] mit Svelte 5 [52, 53] ist ein reaktives JavaScript-Framework, das zur Build-Zeit zu optimiertem Vanilla JavaScript kompiliert wird. Im Gegensatz zu Frameworks wie React

oder Vue verwendet Svelte kein Virtual DOM, sondern manipuliert das DOM direkt, was zu besserer Performance und kleineren Bundle-Größen führt [54].

Während Svelte das Kern-Framework für UI-Komponenten darstellt, erweitert SvelteKit dieses um ein vollständiges Web-Application-Framework. SvelteKit fügt Features hinzu, die für echte Web-Anwendungen benötigt werden wie unter anderem Routing für verschiedene URLs, Server-Side Rendering, API-Endpunkte und file-based Routing. Zusammen bilden sie eine Lösung, bei der Svelte die reaktiven Komponenten und SvelteKit die Infrastruktur für vollständige Webanwendungen bereitstellt. [51]

Das zentrale Konzept von Svelte sind reaktive Variablen, die automatisches Re-Rendering auslösen, wenn sich ihre Werte ändern. Die genutzte Version Svelte 5 führt eine neue Syntax mit Runes ein, die explizite Kontrolle über den Komponentenzustand bieten. `$state()` erstellt reaktive Zustandsvariablen, die bei Änderungen automatisch Updates auslösen. `$props()` definiert Eigenschaften, die von außen an eine Komponente übergeben werden und `$bindable()` ermöglicht eine bidirektionale Datenbindung zwischen Parent- und Child-Komponenten. [55]

Obwohl Svelte vergleichsweise jung ist, hat es bereits breiten Community-Support entwickelt. Dies zeigt sich in der wachsenden Anzahl von UI-Bibliotheken, regelmäßigen Community-Events wie Svelte Summit und SvelteHack sowie einer aktiven Entwicklergemeinschaft, die kontinuierlich neue Tools und Komponenten beiträgt.

Tailwind CSS

Tailwind CSS [56, 57] ist ein Utility-First CSS-Framework, das einen grundlegend anderen Ansatz als traditionelle CSS-Frameworks verfolgt. Anstelle von vorgefertigten Komponenten wie Buttons oder Cards stellt Tailwind atomare CSS-Klassen bereit, die jeweils eine spezifische Styling-Eigenschaft repräsentieren. Entwickler können durch das Kombinieren dieser kleinen, zweckgebundenen Klassen wie `bg-blue-500` für einen blauen Hintergrund, `text-lg` für größeren Text oder `p-4` für Padding direkt im HTML schnell und flexibel Layouts und Designs erstellen, ohne eigene CSS-Dateien schreiben zu müssen. [58]

Code Highlighting

Code Highlighting-Bibliotheken ermöglichen die farbliche Hervorhebung von Syntax-Elementen in verschiedenen Sprachen und Formaten, um die Lesbarkeit und das Verständnis zu verbessern. Beide im Folgenden beschriebenen Bibliotheken unterstützen eine Vielzahl von Formaten und bieten spezielle Erweiterungen für das Turtle-Format, das für die Darstellung von RDF-Daten verwendet wird.

Prism [59, 60] ist eine leichtgewichtige Syntax-Highlighting-Bibliothek, die primär für die read-only Darstellung von Code entwickelt wurde. Die Bibliothek zeichnet sich durch ihre geringe Dateigröße und einfache Integration aus, bietet jedoch keine Bearbeitungsfunktionen. Prism eignet sich daher besonders für Anwendungsfälle, in denen Inhalte nur angezeigt, aber nicht interaktiv bearbeitet werden sollen. [61]

CodeMirror [62, 63] ist eine vollständige Code-Editor-Bibliothek, die sowohl Syntax-Highlighting als auch umfangreiche Bearbeitungsfunktionen in einem integrierten System

vereint. Die aktuelle Version CodeMirror 6 bietet einen modularen Aufbau und erweiterte Features wie Syntax-Validierung, Auto-Completion und Bracket-Matching. [62] Durch Erweiterungen wie codemirror-lang-turtle [64] kann CodeMirror spezifisch für RDF-Formate wie Turtle konfiguriert werden, wodurch eine nahtlose Integration von Anzeige und Bearbeitung ermöglicht wird.

2.6.3 REST

REST (Representational State Transfer) ist ein Architekturstil für webbasierte APIs, der über das HTTP-Protokoll eine standardisierte Kommunikation zwischen Frontend und Backend ermöglicht. REST-APIs verwenden die Standard HTTP-Methoden GET, POST, PUT und DELETE für verschiedene Operationen und strukturieren Daten über ressourcenorientierte URLs. [65]

Die Kommunikation erfolgt stateless, was bedeutet, dass jede Anfrage alle notwendigen Informationen enthält und der Server keine Sitzungsinformationen speichert. JSON dient dabei als Datenformat für Request und Response. [66] Im RDF-Architect stellt die REST-API die Verbindung zwischen dem Svelte-Frontend und dem Java Spring-Backend her und ermöglicht den Austausch von Daten über spezifische Endpunkte.

3 Anforderungen und Forschungsfragen

In diesem Kapitel werden die Anforderungen an die zu implementierende SHACL-Erweiterung für den RDF-Architect und die daraus resultierenden Forschungsfragen formuliert.

3.1 Anforderungskatalog

Die Anforderungen sind thematisch nach ihren Funktionsbereichen aufgeteilt und jeweils nach Muss-, Soll- und Kann-Anforderungen kategorisiert.

Die Kernfunktionalitäten der zu implementierenden SHACL-Erweiterung des RDF-Architect Systems sind die Generierung der SHACL-Regeln und die kontextuelle Visualisierung von SHACL-Shapes.

Die Implementierung **muss** in das RDF-Architect-Projekt integriert werden und entsprechend der bestehenden Architektur die Apache Jena RDF API als Framework für das Arbeiten mit RDF-Daten nutzen.

Das System **muss** in der Lage sein, aus CIM/XML-Ontologien, welche nach dem CGMES 3.0 Standard definiert sind, automatisch passende SHACL-Shapes abzuleiten.

Zunächst **muss** analysiert werden, welche SHACL-Shapes in den ENTSO-E-Veröffentlichungen für CGMES 3.0 definiert sind und welche davon aus dem zugehörigen CIM-Modell generierbar sind, ohne dass ergänzende Informationen benötigt werden. Es **müssen** außerdem die Shapes identifiziert werden, für welche dies nicht möglich ist.

Für alle instanziierbaren Klassen, der ENTSO-E Veröffentlichungen, **müssen** anschließend SHACL-Shapes generiert werden, welche Instanzdaten für Klassenattribute und Assoziationen validieren. Das Validieren der Instanzdaten eines Attributes bezieht sich darauf, dass überprüft wird, ob der Datentyp des Wertes korrekt ist und dass die vorgegebene Kardinalität eingehalten wird. Bei einer Assoziation wird ähnlich hierzu überprüft, ob es sich bei dem Ziel der Assoziation um eine Instanz der korrekten Klasse handelt und ebenfalls ob die Kardinalität eingehalten wird.

Es **muss** außerdem die Vererbungshierarchie der Klassen berücksichtigt werden. Bei dieser Vererbungshierarchie werden Attribute und Assoziationen an erbende Klassen weitergegeben. Diese Properties **müssen** sowohl für vererbende Klasse als auch für die erbenden Klassen validiert werden.

Um die Validität der generierten Shapes zu gewährleisten, **müssen** Tests geschrieben werden, welche die korrekte Funktionsweise der SHACL-Generierung anhand repräsentativer CIM-Daten überprüfen. Da der RDF-Architect auf CGMES 3.0 basiert, genügen hier als Testdaten mit diesem erstellte Beispieldaten. Zugehörige Instanzdaten können sowohl für Positiv- als auch Negativfälle manuell geschrieben werden.

Neben der Generierung von SHACL-Regeln **muss** das System zusätzlich die Möglichkeit bieten, SHACL-Regeln zu importieren. Das Format RDF/XML **muss** unterstützt werden. Optional **können** auch weitere RDF-Formate wie TURTLE, N-Triples oder JSON-LD unterstützt werden. Der Import bedeutet, dass der in der Datei enthaltene Graph hochgeladen und abgespeichert wird.

Hochgeladene SHACL-Dateien **müssen** in-Memory gespeichert werden. Ein wiederholtes Importieren **muss** den letzten Import überschreiben. Hier ersetzt der neue Import den alten, wobei dieser unwiderruflich gelöscht wird. Importierte Dateien **sollen** zugehörig zu einer Ontologie abgespeichert werden. Zugehörig bedeutet, dass es eine 1 zu 1 Relation zwischen Ontologie und SHACL-Import gibt.

Sowohl importierte als auch generierte Shapes **müssen** exportierbar sein. Dieser Export muss über einen Dateidownload geschehen. Es **muss** möglich sein, importierte und generierte Shapes separat zu exportieren. Es **kann** zusätzlich die Möglichkeit geben, diese zusammengeführt in einer Datei zu exportieren. Hier **muss** wieder das RDF/XML-Datenformat als unterstütztes Exportformat angeboten werden. Es **können** außerdem weitere Formate wie beim Import unterstützt werden.

Um SHACL-Shapes benutzerfreundlich im RDF-Architect zu visualisieren, **muss** das System generierte und importierte SHACL-Shapes den jeweils zu validierenden Eigenschaften der Klasse im UML-Klassendiagramm des RDF-Architects zuordnen können. Diese Eigenschaften umfassen Attribute und Assoziationen. Durch diese Zuordnung **muss** es die Möglichkeit geben, bei Betrachtung eines Attributes oder einer Assoziation im RDF-Architect die zugehörigen SHACL-Shapes anzuzeigen. Das Navigieren der Erweiterung **soll** auch ohne weitere SHACL-Kenntnisse möglich sein. Teil der Arbeit ist die Entwicklung einer Lösung hierfür. Denkbar ist ein Fenster, welches kontextspezifische SHACL-Shapes anzeigt, je nachdem von welcher Ressource, also welchem Attribut, welcher Assoziation oder Klasse es heraus geöffnet wurde. Es **können** außerdem Visualisierungstechniken zur Verbesserung der Benutzerfreundlichkeit eingesetzt werden.

3.2 Forschungsfragen

Aus den definierten Anforderungen ergeben sich die folgenden zentralen Forschungsfragen, die den Fokus dieser Arbeit bestimmen:

F1: Reichen die von der ENTSO-E für den CGMES 3.0 Standard veröffentlichten SHACL-Shapes für eine vollständige Validierung aus und inwieweit können diese durch eine automatische Generierung abgedeckt werden?

Hierdurch sollen die Möglichkeiten und Grenzen der automatischen SHACL-Generierung aufgezeigt werden. Es soll systematisch untersucht werden, welche Validierungsregeln sich aus den CIM/XML-Ontologien ableiten lassen. Die Überprüfung dient außerdem der praktischen Validierung des entwickelten Ansatzes anhand der für den RDF-Architect relevanten Industriestandards und zeigt die Praxistauglichkeit der automatischen Generierung auf. Es soll außerdem auf eventuelle Lücken in den veröffentlichten SHACL-Regeln aufmerksam gemacht werden.

F2: Wie kann die Integration von SHACL-Funktionalitäten in den RDF-Architect gestaltet werden, so dass auch Benutzer ohne SHACL-Expertise effektiv damit arbeiten können?

Hier soll vor allem gezeigt werden, wie SHACL-Funktionalitäten in den bestehenden Modellierungsworkflow integriert werden können und wie eine solche Visualisierung auch ohne erweiterte SHACL-Kenntnisse verständlich ist.

4 Konzeption der SHACL-Erweiterung für den RDF-Architect UML-Editor

4.1 Analyse existierender SHACL-Dateien

Die ENTSO-E hat ergänzend zum CGMES 3.0 Standard zugehörige SHACL-Validierungsdateien veröffentlicht. Da die Kompatibilität mit CGMES 3.0 eine zentrale Anforderung für den RDF-Architect darstellt, wurden diese SHACL-Dateien vor der eigentlichen Konzeption der Lösung einer detaillierten Analyse unterzogen. Der Analysefokus lag auf der Identifikation existierender Shape-Typen, deren Validierungsfunktionen sowie der Evaluierung ihrer automatischen Generierbarkeit. Ein weiteres zentrales Ziel war die Entwicklung einer Zuordnungslogik zwischen SHACL-Shapes und Ontologie-Elementen, um die spätere Visualisierung der Shapes im RDF-Architect zu ermöglichen.

Im Falle, dass sich nicht direkt erschließen ließ, was etwaige SHACL-Prädikate bedeuten, wurde die W3C Recommendation zur SHACL herangezogen. [27]

Im Detail wurden die Namenskonventionen, die enthaltenen Triples der einzelnen Shapes, deren Validierungslogik und der Zusammenhang zu den zugrundeliegenden Ontologie-Informationen untersucht. Es ist zu beachten, dass die veröffentlichten Shapes vermutlich aus einer Kombination von automatischer Generierung und manueller Nachbearbeitung entstanden sind, wodurch Inkonsistenzen zwischen den Shapes und den zugrundeliegenden Ontologien existieren können.

Die ENTSO-E stellt die SHACL-Dateien in einer strukturierten Verzeichnishierarchie zur Verfügung. Für die Analyse wurden zuerst die SHACL-Dateien im RDFS-Ordner untersucht, da diese die Kernvalidierungen für die strukturellen Eigenschaften der CGMES 3.0 Profile enthalten. Es wurde das "DiagramLayoutProfile" unter der Annahme ausgewählt, dass alle weiteren Shapes im "RDFS" Ordner nach dem gleichen Prinzip erstellt wurden. Im Anschluss an die Analyse wurde diese Vermutung noch einmal überprüft. Die weiteren Verzeichnisse wurden dann ebenfalls betrachtet, um ein vollständiges Bild der existierenden Validierungsregeln zu erhalten.

Ein durchgängiges Muster in der SHACL-Datei ist die Verwendung eines profilspezifischen Namespace-Präfixes. Während die Ontologie-Elemente den Standard-CIM-Präfix `cim:` verwenden, nutzen alle generierten SHACL-Ressourcen ein eigenes Präfix. Im Fall des DiagramLayout-Profils ist dies `dl:`, welches die IRI `http://iec.ch/TC57/ns/CIM/DiagramLayout-EU/Constraints#` subsituiert. Diese Präfix-Änderung ermöglicht eine klare Trennung zwischen Ontologie- und Validierungselementen.

Der erste Analyseschritt fokussierte sich auf die Erfassung aller SHACL-Ressourcen-Typen. Mittels einer SPARQL-Query (siehe Anhang 8.1.1) wurden in der "RDFS" SHACL-Datei "DiagramLayoutProfile.ttl" für das DiagramLayout-Profil 69 Ressourcen identifiziert, die sich wie folgt aufteilen: 11 NodeShapes, 52 PropertyShapes, 5 PropertyGroups sowie eine Ontologie-Referenz. Die identifizierten Ressourcen, wurden im Folgenden auf ihre Funktion und automatische Generierbarkeit untersucht.

4.1.1 owl:Ontology

Die `owl:Ontology` Ressource enthält umfangreiche Metadaten zur Validierungsdatei selbst. Hier werden verschiedene Aspekte der SHACL-Datei dokumentiert, unter anderem die Kompatibilität zu relevanten IEC-Standards. Weiterhin werden Versionsinformationen sowohl in maschinenlesbarer als auch menschenlesbarer Form angegeben. Die Ressource enthält außerdem Angaben zum Erstellungs- und Änderungszeitpunkt der Datei sowie Informationen über den Herausgeber und Ersteller.

Diese Metadaten sind kontextspezifisch und nicht relevant für die SHACL-Validierung. Sie sind nicht ableitbar aus der zugrundeliegenden CIM/XML-Ontologie. Für eine automatische Generierung dieser Metadaten im Rahmen der Generierung der SHACL-Regeln wären Informationen wie etwa die aktuelle Versionsnummer des Projekts, der Benutzer oder die verwendeten Konformitätsstandards notwendig. Der RDF-Architect müsste hierfür über eine Projekt- und Versionsverwaltung verfügen, die diese Metadaten verwaltet und bei der Generierung bereitstellt. Da diese Funktionalität aktuell nicht existiert, wäre eine automatische Generierung dieser Ressource nur sehr begrenzt möglich.

4.1.2 sh:NodeShape

Im zweiten Analyseschritt wurde die Beziehung zwischen NodeShapes und CIM-Klassen untersucht. NodeShapes verbinden die Ontology mit den PropertyShapes. Sie sammeln mittels `sh:property` zugehörige PropertyShapes und verweisen über das `sh:targetClass` Prädikat auf die Klasse, auf deren Instanzen diese angewendet werden sollen. Die Benennung der NodeShapes folgt einer einfachen Konvention: Sie übernehmen die IRI der zu validierenden Klasse, ersetzen jedoch das Namespace-Präfix. So wird beispielsweise aus der Klasse `cim:DiagramObject` das NodeShape `dl:DiagramObject`. Alle NodeShapes sind durch ein Statement `rdf:type` `sh:NodeShape` als solche gekennzeichnet. Hieraus folgt, dass unter der Bedingung, dass es für die später generierten Shapes ein eigenes Präfix gibt, diese Namenskonvention übernommen werden kann.

Bei der quantitativen Analyse (siehe Anhang 8.1.2) fällt auf, dass das DiagramLayout-Profil zwar 18 Klassen enthält, aber nur 11 NodeShapes für nur 8 dieser Klassen definiert sind. Diese ungleiche Verteilung lässt sich folgendermaßen erklären: Einige Klassen verfügen über mehrere NodeShapes, die sich problemlos zusammenführen lassen würden und somit redundant sind. Andere Klassen hingegen haben gar keine NodeShapes, was sich durch spezifische Generierungskriterien erklärt. Ein NodeShape existiert nur für instanziierbare Klassen mit validierbaren Eigenschaften. Die Instanziierbarkeit erfordert das Vorhandensein des `cims:stereotype` `http://iec.ch/TC57/NonStandard/UML#concrete` bei gleichzeitiger Abwesenheit ausschließender Stereotypen. Im DiagramLayout-Profil sind dies `enumeration` und `CIMDatatype`, während in anderen CGMES-Profilen wie `Dynamics` zusätzlich der Stereotyp `Description` zum Ausschluss führt. Zu den validierbaren Eigenschaften zählen sowohl direkte als auch geerbte Attribute sowie Assoziationen. Für den Fall, dass es weitere Kriterien für die Instanziierbarkeit oder auch Nicht-Instanziierbarkeit einer Klasse gibt, soll bei der Implementierung darauf geachtet werden, dass eine neue Erkenntnis keine umfangreichen Anpassungen nach sich zieht.

4.1.3 sh:PropertyShape

Im nächsten Schritt wurden die PropertyShapes und deren Zusammenhang zur Ontologie genauer untersucht. Bei der Betrachtung der Triples fällt auf, dass sich die Shapes über das Prädikat `sh:path` entweder einem Attribut oder einer Assoziation zuordnen lassen. Eine SPARQL-Abfrage nach den IRIs aller PropertyShapes zeigt, dass drei verschiedene PropertyShape-Typen existieren, die durch die folgenden Suffixe gekennzeichnet sind: `-cardinality`, `-datatype` und `-valueType`. Voran steht immer die IRI mit zum SHACL-Präfix geändertem Präfix.

Die Analyse der `-cardinality` PropertyShapes ergab, dass diese immer mindestens einen der beiden Constraints `sh:minCount` oder `sh:maxCount` aufweisen. Dies bestätigt zusammen mit der sprechenden Namensgebung ihre Funktion, dass sie die Kardinalität, d.h. die Anzahl der zulässigen Vorkommen einer Property validieren. Die beiden anderen Typen validieren Datentypen, wobei zunächst unklar war, was diese unterscheidet.

Die Hypothese, dass `-datatype` Shapes für Attribute und `-valueType` Shapes für Assoziationen verwendet werden, konnte durch eine weitere SPARQL-Abfrage (siehe Anhang 8.1.3) bestätigt werden. Die Ergebnisse zeigen eindeutig: `-cardinality` Shapes existieren in den ENSTO-E Dateien des DiagramLayout-Profils, sowohl für Attribute als auch für Assoziationen, während `-datatype` PropertyShapes ausschließlich für Attribute und `-valueType` Shapes nur für Assoziationen verwendet werden.

Eine quantitative Analyse der zugrundeliegenden Attribut-PropertyShapes lieferte folgende Erkenntnisse: Von 23 existierenden Attributen in der zugrundeliegenden CIM-Ontologie werden 19 in SHACL referenziert, wobei für jedes referenzierte Attribut exakt zwei PropertyShapes vorliegen werden: eines mit dem Suffix `-datatype` und eines mit `-cardinality`. Dies erklärt die Gesamtzahl von 38 PropertyShapes. Die vier nicht referenzierten Attribute gehören zu nicht instanziierbaren Klassen ohne instanziierbare Unterklassen, was konsistent mit den bereits identifizierten Generierungskriterien ist.

Eine analoge Analyse für Assoziations-PropertyShapes ergab, dass auch hier für jede referenzierte Assoziation exakt zwei PropertyShapes existieren: eine `-valueType` und eine `-cardinality` PropertyShape.

Folglich müssen bei der Generierung für jedes Attribut und jede Assoziation, mit Zugehörigkeit zu einer instanziierbaren Klasse oder einer Klasse mit instanziierbarer Unterklasse, zwei PropertyShapes erstellt werden: für beide die `-cardinality` PropertyShape und im Falle eines Attributs zusätzlich ein `-datatype` und im Falle einer Assoziation ein `-valueType` PropertyShape.

Im Folgenden wurden die Triple der PropertyShapes genauer betrachtet. Eine SPARQL-Query identifizierte die Prädikate, die alle PropertyShapes gemeinsam haben: `rdf:type`, `sh:severity`, `sh:name`, `sh:order`, `sh:message`, `sh:description`, `sh:group` und `sh:path`. Ein PropertyShape könnte beispielsweise wie in Listing 9 gezeigt, aussehen und enthält außerdem die Statements `sh:datatype xsd:float` und `sh:nodeKind sh:Literal`, welche spezifisch für dieses PropertyShape sind.

```
dl:DiagramObjectPoint.xPosition-datatype
    rdf:type          sh:PropertyShape;
    sh:description    "This constraint validates the datatype of the property (attribute).";
    sh:message        "The datatype is not literal or it violates the xsd datatype.";
    sh:group          dl:DatatypesGroup;
    sh:name           "DiagramObjectPoint.xPosition-datatype";
    sh:order          2;
    sh:path           cim:DiagramObjectPoint.xPosition;
    sh:severity       sh:Violation;
    sh:datatype       xsd:float;
    sh:nodeKind       sh:Literal.
```

Listing 9: veröffentlichtes -datatype PropertyShape

Der `rdf:type` definiert bei allen Shapes deren Typ als `sh:PropertyShape`. Die `sh:severity` ist durchgehend auf `sh:Violation` gesetzt, wodurch sichergestellt wird, dass Regelverletzungen die Daten als invalide kennzeichnen. Der `sh:name` entspricht dem Label der Property mit dem zugehörigen Suffix. Die `sh:order` wird so festgelegt, dass die Eigenschaften einer Klasse alphabetisch sortiert und durchnummeriert werden, wobei jedes PropertyShape die Nummer der zugehörigen Eigenschaft erhält. Eine Besonderheit stellen die IdentifiedObject-Eigenschaften dar: Diese erhalten die Order 0.1 und stehen somit immer an erster Stelle. Die `sh:message` enthält eine kurze Fehlermeldung für Regelverletzungen, während `sh:description` eine Beschreibung davon bereitstellt, was das PropertyShape validiert. Das Prädikat `sh:path` definiert, welche Property das PropertyShape validieren soll, und `sh:group` ordnet das PropertyShape einer PropertyGroup zu. Der zu entwickelnde Generator soll diesem Prinzip folgen. Aufgrund des domänenspezifischen Kontextes soll es jedoch keinen Sonderfall für "IdentifiedObject" geben.

Die `-datatype` Shapes lassen sich in zwei Kategorien unterteilen: Shapes für Attribute, deren CIM-Datentyp sich in einen XSD-Datentyp übersetzen lässt, und Shapes für Attribute mit Enum-Datentypen. Bei Attributen, deren CIM-Datentyp in XSD-Datentypen übersetzbar ist, wie `cim:String`, `cim:Float` oder `cim:Boolean`, wird ein Triple mit dem Prädikat `sh:datatype` und dem entsprechenden XSD-Datentyp als Objekt genutzt. Ein weiterer Sonderfall ist ein CIM-Datentyp, welcher den Stereotyp `CIMDatatype` besitzt. Hier wird der CIM-Datentyp des "value"-Attributs der CIMDatatype-Klasse als Datentyp genutzt. Dass dieses "value"-Attribut für CIMDatatype-Klassen existiert, ist im gesamten CGMES 3.0 konsistent und wird daher als universell anwendbar gesehen. Zusätzlich wird `sh:nodeKind` auf `sh:Literal` gesetzt. Der Instanzwert muss folglich ein Literal sein, das dem XSD-Datentyp entspricht. Bei Enum-Datentypen wird stattdessen ein `sh:in`-Statement mit einer Liste aller zugehörigen Enum-Eintrag-IRIs als Objekt verwendet. In diesem Fall wird `sh:nodeKind` auf `sh:IRI` gesetzt, wodurch der Instanzwert eine der aufgelisteten IRIs sein muss. Diese Zusammenhänge sollen auch im Generator genauso umgesetzt werden.

Die `-valueType` PropertyShapes verifizieren den Datentyp der referenzierten Klasse auf andere Art. Hier wird zwar auch ein `sh:in` Prädikat genutzt, welches die erlaubten Klassen auflistet und mittels eines `sh:nodeKind` Prädikat eine `sh:IRI` voraussetzt. Jedoch wird hier eine IRI erwartet, welche eine Instanz einer Klasse ist. Um diese auf den richtigen Typ zu prüfen, führt das Prädikat `sh:path` nicht direkt zu einer zu verifizierenden IRI. Hier wird stattdessen eine Liste an Pfaden angegeben. In diesem Fall besteht sie aus der Assoziations-IRI, welche in den Instanzdaten auf eine Klasseninstanz zeigt und dann via dem Prädikat

rdf:type wiederum auf eine CIM-Klasse, welche in den erlaubten Klassen vorkommen muss.

Listing 10 ist dazu ein exemplarisches Bespiel, welches eine Assoziation von einer Klasse cim:Class1 zu einer cim:Class2 beschreibt, sowie ein zugehöriges Instanz-Beispiel. In diesem Fall wäre das Objekt von sh:path dann (cim:Class1.Class2 rdf:type). Es wird in zwei Schritten von der :instanz1 auf cim:Class2 verwiesen. Zuerst ausgehend von :instanz1 über cim:Class1.Class2 auf :instanz2 und dann von dieser über rdf:type auf cim:Class2. Diese IRI wird dann auf Vorkommen in der Liste des Objekts des Prädikats sh:in überprüft, welche für dieses Beispiel nur die IRI der Klasse cim:Class2 enthalten würde.

```
//Ontology
cim:Class1
    rdf:type        rdf:Class ;
    rdfs:label      "Class1"@en;
    cims:stereotype <http://iec.ch/TC57/NonStandard/UML#concrete> .

cim:Class2
    rdf:type        rdf:Class ;
    rdfs:label      "Class2"@en;
    cims:stereotype <http://iec.ch/TC57/NonStandard/UML#concrete> .

cim:Class2.Class1
    rdf:type              rdf:Property;
    rdfs:domain           cim:Class2;
    rdfs:range            cim:Class1;
    rdfs:label            "Class2.Class1"@en;
    cims:multiplicity      cims:M:0..1;
    cims:inverseRoleName   cim:Class1.Class2;
    cims:associationUsed   "Yes".

cim:Class1.Class2
    rdf:type               rdf:Property;
    rdfs:domain            cim:Class1;
    rdfs:range             cim:Class2;
    rdfs:label             "Class1.Class2"@en;
    cims:multiplicity      cims:M:n;
    cims:inverseRoleName   cim:Class2.Class1;
    cims:associationUsed   "No".

//Instance
:instanz1 rdf:type cim:Class1;
          cim:Class1.Class2 :instanz2.
:instanz2 rdf:type cim:Class2.
```

Listing 10: Instanzbeispiel Assoziation

Dieses PropertyShape hat jedoch eine Schwachstelle. Ist eine referenzierte Instanz nicht typisiert, wird das -valueType PropertyShape nicht ausgelöst. Das bedeutet im Beispiel aus Listing 10, dass falls :instanz2 kein rdf:type Prädikat hätte, keine Violation ausgelöst würde.

Die -cardinality Shapes sind für Attribute und Assoziationen einheitlich. Wie in der Analyse bereits festgestellt, wird die Anzahl der Vorkommen über sh:minCount und

sh:maxCount verifiziert. Diese leiten sich direkt aus dem Lower- und UpperBound des Objekts des Prädikats cims:multiplicity der CIM-Ressource ab. Hier gibt es jedoch auch wieder einen Sonderfall. Ist der LowerBound 0 und der UpperBound beliebig, gibt es keine zu verifizierende Kardinalität. In diesem Fall existiert keine -cardinality Shape, sondern nur eine für die Typvalidierung der Property.

4.1.4 sh:PropertyGroups

Die SHACL PropertyGroups organisieren PropertyShapes nach ihrer funktionalen Zugehörigkeit. Fünf Gruppen strukturieren die PropertyShapes: dl:CardinalityGroup für Kardinalitäts-Shapes, dl:DatatypesGroup für Datentyp-Validierungen, dl:AssociationsGroup für Referenz-Prüfungen sowie die beiden IdentifiedObject-spezifischen Gruppen ido:DatatypesGroupIO und ido:CardinalityGroup. Die separate Behandlung von "IdentifiedObject"-PropertyShapes durch eigene PropertyGroups unterstreicht die besondere Bedeutung dieser Basisklasse im CIM-Kontext und ermöglicht eine differenzierte Validierungsstrategie.

4.1.5 Betrachtung weiterer SHACL-Dateien

Diese Erkenntnisse konnten durch die Anwendung derselben SPARQL-Queries auf alle Profile im RDFS-Ordner und deren CIM-Ontologien bestätigt werden.

Im Folgenden wurden dann weitere SHACL-Dateien aus der Veröffentlichung der ENTSO-E betrachtet.

Im Ordner "Common" liegt nur eine Datei mit dem Namen "IdentifiedObjectStringLength.ttl". Hier wird durch die SHACL Regeln geprüft, ob bestimmte String-Properties des "IdentifiedObjects" maximale oder exakte Zeichenlängen einhalten. Diese Längeneinschränkungen sind an Definitionen der ENTSO-E gekoppelt, welche nur kommerziell erworben werden können und für diese Arbeit nicht zugänglich waren. Da auch im RDF-Architect keine Informationen über diese Definitionen und die zugehörigen Längeneinschränkungen vorliegen, können diese Shapes dort auch nicht automatisiert generiert werden.

Im Ordner "CrossProfileValueType" liegen Shapes zur Validierung profilübergreifender -valueType Regeln. Da der RDF-Architect zum jetzigen Stand jedoch keine Funktionen zur Verwaltung mehrerer zusammenhängender Profile enthält, können diese Shapes momentan nicht automatisch generiert werden.

Der Ordner "DatatypeMapping" enthält eine Datei, welche einem CIM-Attribute einen XSD- und Java-Datentyp zuweist. Da dieses Mapping manuell definiert werden müsste, fällt es aus der automatischen Generierung heraus. Ein Konzept für das Mapping von CIM-Datentyp zu XSD-Datentyp wird in Kapitel 4.2 erstellt.

Die SHACL-Dateien in den IEC-Verzeichnissen enthalten Regeln zur Einhaltung der IEC-Standards selbst. Hier werden Vorgaben wie Namenskonventionen und Modellierungsrichtlinien, welche nicht aus der CIM/XML-Ontologie ableitbar sind, validiert.

Der "Main"-Ordner beinhaltet nur eine Datei, die die anderen SHACL-Dateien importiert. Hier werden dementsprechend keine eigenen Shapes definiert. Da die Generierung auf eine

einzelne SHACL-Datei beschränkt ist, ist eine Generierung einer solchen, andere SHACL Dateien importierenden Datei nicht notwendig.

Die in den Dateien in den Ordnern "SolvedMAS" und "NotSolvedMAS" enthaltenen SHACL-Shapes enthalten spezifische energiewirtschaftliche Validierungslogik. Diese Regeln prüfen beispielsweise, ob berechnete Spannungswerte nach einer Lastflussberechnung in physikalisch plausiblen Bereichen liegen. Die PropertyShapes nutzen komplexe SPARQL-Queries, die Zusammenhänge zwischen verschiedenen Netzkomponenten analysieren. Aufgrund dieser Abhängigkeit von domänenspezifischem Wissen, das über strukturelle UML-Eigenschaften hinausgeht, können diese Validierungsregeln nicht automatisch generiert werden.

Zuletzt wurden noch die Dateien im Ordner "InverseAssocCardinality" betrachtet. Dateien in diesem Ordner beinhalten SHACL Regeln zum Validieren der Multiplizität der Rückrichtung einer Assoziation. Assoziationen haben in der CIM-Darstellung immer zwei Ressourcen, welche eine Richtung der Assoziation beschreiben. Im Standardfall zeigt eine Assoziation in eine Richtung und hat eine Multiplizität von etwas Bestimmtem zu etwas Unbestimmtem. Zum Beispiel 1 bis 1 und in der Rückrichtung 0 bis n. In gewissen Fällen ist diese inverse Multiplizität jedoch auch bestimmt. Für diesen Fall existieren die hier betrachteten `-InverseCardinality`-Shapes. Es wird somit validiert, wie oft eine Klasse durch eine Assoziation referenziert werden darf. Diese Regel ist automatisiert generierbar und soll in der Konzeption und Implementierung berücksichtigt werden. Anders als bei der Validierung in die direkte Richtung einer Assoziation sind hier keine weiteren PropertyShapes zur inversen Validierung des Datentyps vorhanden, da diese durch die direkte `-datatype` PropertyShape bereits implizit abgedeckt sind.

Eine solche mit dem Suffix `-InverseCardinality` gekennzeichnete PropertyShape wird sehr ähnlich der einfachen `-cardinality` PropertyShape erstellt. Der Unterschied besteht darin, dass diese Regel mittels `sh:path` auf eine Blank Node zeigt, welche wiederum mittels `sh:inversePath` über den Pfad der nicht-inversen Assoziation auf die Klasse verweist, von welcher die Assoziation ausgeht.

4.2 SHACL-Generierung aus CIM/XML

Um eine konsistente und vollständige SHACL-Generierung zu gewährleisten, definiert das folgende Konzept Regeln für die Überführung des CIM-Modells zu SHACL-Regeln. Die Generierung folgt einem ressourcenorientierten Ansatz, bei dem jede CIM-Ressource betrachtet und dann in entsprechende SHACL-Regeln überführt wird. Das Konzept orientiert sich im Wesentlichen an den Ergebnissen der voranstehenden Analyse existierender SHACL-Regeln und modifiziert und erweitert diese. Im Folgenden wird beschrieben, unter welchen Regeln die Überführung der CIM-Ressourcen in SHACL-Shapes genau geschehen soll.

4.2.1 CIM-Property zu PropertyShapes

Die Aufteilung von CIM-Properties in Attribute und Assoziationen wird auch hier berücksichtigt. Unabhängig davon werden die Properties alphabetisch sortiert und durchnummeriert. Ihnen wird eine `sh:order` zugeteilt, welche für eine Sortierung genutzt werden kann. Diese verweist auf eine Floating Point Zahl, welche von 0.0 ausgehend ganzzahlig hochzählt. Jedes PropertyShape wird außerdem einer PropertyGroup zugeordnet.

Die IRI und der sh:name werden für jede Art von PropertyShape gleich erstellt. Um den sh:name zu bestimmen, wird erst das rdfs:label der Property separiert durch einen Punkt an den Klassennamen, zu welcher die Property gehört, angehangen. Ein zusätzlich hinzugefügtes Suffix beschreibt, was das PropertyShape validiert. Die IRI ist so der an das SHACL-Präfix angehängte sh:name des PropertyShapes. Dieses Präfix ist universell für alle generierten SHACL-Regeln und fest gesetzt auf rdfash:, was für RDF-Architect-SHACL steht. Die zu dem Präfix gehörende IRI ist: http://example.com/shacl#. Für ein PropertyShape mit den Suffix -datatype eines Attributes mit dem rdfs:label "xPosition" und einer Klasse mit dem rdfs:label "DiagramObjectPoint" wird beispielsweise die IRI rdfash:DiagramObjectPoint.xPosition-datatype erstellt.

Für die an den Ergebnissen der Analyse orientierten PropertyShapes gilt, sowie dort auch festgestellt, dass die sh:severity auf sh:Violation gesetzt wird. Ein Verstoß gegen die Regel stuft die Instanzdaten folglich als invalide ein. Die PropertyShapes werden außerdem durch die Relation rdf:type sh:PropertyShape als solche markiert.

Cardinality PropertyShape für alle Properties

Unabhängig davon, ob es sich um ein Attribut oder eine Assoziation handelt, wird für jede Property eine PropertyShape mit dem Suffix -cardinality erstellt, falls die Kardinalität für diese einschränkend ist. Um dieses PropertyShape zu erstellen, muss die cims:multiplicity ausgelesen werden. Aus dieser werden die Constraints sh:minCount mit dem lowerBound als Objekt und sh:maxCount mit dem upperBound als Objekt erzeugt. Falls jedoch der lowerBound gleich null oder der upperBound unbestimmt ist, werden die jeweiligen Constraints ausgelassen. Im Falle, dass beide Constraints ausgelassen werden, wird das PropertyShape im Ganzen nicht erstellt, da es ohne Constraints nichts validiert und somit keinen Mehrwert für die Validierung bietet.

Neben den Constraints und den für alle PropertyShapes zu erstellenden Triples wird außerdem eine sh:description für eine Beschreibung dessen, was validiert wird, die Zuordnung zur rdfash:CardinalityGroup über das sh:group Prädikat, eine sh:message mit einer Fehlermeldung und einem sh:path Prädikat, welche die IRI der Property als Objekt hat erstellt. Eine -cardinality PropertyShape mit einer Multiplizität von zwei bis drei würde dementsprechend, wie in Listing 11 dargestellt, aussehen.

```
rdfash:KlassenName.PropertyLabel-cardinality
    rdf:type          sh:PropertyShape;
    sh:description  "This constraint validates the cardinality of the property (<<propertyType>>).";
    sh:group          rdfash:CardinalityGroup;
    sh:minCount       2;
    sh:maxCount       3;
    sh:message        "Cardinality violation 2..3 (<<propertyType>>).";
    sh:name           "KlassenName.PropertyLabel-cardinality";
    sh:order          Die Order der Property(z.B. 1.0);
    sh:path           [attributeIRI];
    sh:severity       sh:Violation .
```

Listing 11: Muster -cardinality PropertyShape

Die PropertyShape aus Listing 11 meldet dementsprechend eine Violation, wenn es die Property weniger als zwei oder mehr als dreimal gibt. Der Platzhalter `<<propertyType>>` wird entweder durch `attribute` oder `association` ersetzt, je nachdem, was die zugrundeliegende Property ist. Die `sh:message` steht in Abhängigkeit zu der der Kardinalität. Ist der lowerBound null, lautet diese "`Cardinality violation. Upper bound shall be <<upperBound>> (<<propertyType>>).`". Ist der upper Bound im Gegensatz zum lowerbound nicht definiert, lautet die `sh:message` hingegen "`Missing required property (<<propertyType>>).`". Im Fall, dass sowohl lower- als auch upperBound definiert sind, ist die `sh:message` dann "`Cardinality violation <<lowerBound>>..<<upperBound>> (<<propertyType>>).`". Diese Fehlermeldungen sollen ein einfaches Verständnis dafür bieten, welcher Fehler aufgetreten ist und dem Nutzer die Arbeit abnehmen, danach zu suchen, ob diese Property zu oft oder zu selten existiert.

Attribute zu PropertyShapes

Ein Attribut ist im CIM dadurch charakterisiert, dass es neben der `rdf:type rdf:Property` Relation den `cims:stereotype http://iec.ch/TC57/NonStandard/UML#attribute` hat. Wie bereits in der Analyse festgestellt, können zwei verschiedene PropertyShapes erstellt werden, um ein Attribut zu validieren: einmal eine `-datatype` und einmal die bereits beschriebene `-cardinality` PropertyShape.

Für die Erstellung der `-datatype` PropertyShape müssen zusätzlich zu den für den Namen benötigten Informationen noch weitere über den Datentyp ausgelesen werden. Diese sind entweder das Objekt des Prädikats `cims:dataType` oder `rdfs:range`, von denen immer eines vorhanden ist. `cim:datatype` zeigt zu einem Datentypen. `rdfs:range` hingegen wird verwendet, um eine Liste an erlaubten Werten anzugeben. Dies wird in diesem Kontext für Enumerationen genutzt. Unabhängig davon, ob es sich um ein Attribut mit `cims:dataType` oder `rdfs:range` handelt, werden die Triple wie in Listing 12 ersichtlich erstellt.

```
rdfash:Klassenlabel.AttributLabel-datatype
    rdf:type          sh:PropertyShape;
    sh:description    "This constraint validates the datatype of the property (attribute).";
    sh:group          rdfash:DatatypesGroup;
    sh:name           "Klassenlabel.AttributLabel-datatype";
    sh:order          Die Order der Property(z.B. 1.0);
    sh:path           [attributeIRI];
    sh:severity       sh:Violation .
```

Listing 12: Muster -datatype base-PropertyShape

Bei Auftreten des Prädikat `cims:dataType`, ist das Objekt der Relation, im CGMES 3.0, eine Klasse, welche einen XSD-Datentypen widerspiegelt. Im RDF-Architect wird die Möglichkeit geboten, dass hier auch direkt ein XSD-Datentyp steht. In diesem Fall, wird dieser direkt übernommen. Zeigt das Prädikat jedoch zu einer Klasse, ist diese durch den

cims:stereotype Primitive, RDF oder CIMDatatype gekennzeichnet. Im Falle der Primitive Klasse wird die Zuordnung, welchen XSD-Datentypen die Klasse darstellt, über das rdfs:label der Klasse festgelegt. Es wird versucht, einen XSD-Datentyp mit identischem Namen zu finden. Um die Einheitlichkeit zu erhöhen, ohne eine Mehrdeutigkeit zu erzeugen, soll diese Zuordnung case-insensitive sein. Eine CIM-Klasse mit dem cims:stereotype Primitive und dem rdfs:label Float wird folglich auf den XSD-Datentyp xsd:float gemappt. Ist diese Zuordnung nicht möglich, wird die IRI der als Datentyp genutzten Klasse als Datentyp gesetzt. Im Falle einer CIMDatatype Klasse, hat diese Klasse nach den CGMES 3.0 Daten immer ein Attribut mit dem rdfs:label value. Dieses zeigt mittels cim:dataType wiederum auf eine Primitive Klasse, bei welcher das oben genannte Mapping anwendet werden kann. Falls die Klasse den Stereotyp RDF besitzt, handelt es sich bei dem Datentyp um ein Statement. Dieses muss als XML-Literal abgespeichert werden, weshalb dieser als zu validierender Datentyp genutzt. In diesem Fall werden die Statements sh:datatype, welches auf einen XSD-Datentypen zeigt, sh:nodeKind sh:Literal und eine sh:message, welche eine Fehlermeldung enthält, erstellt. Kann der Datentyp des Attributs beispielsweise auf den Datentyp xsd:float gemappt werden, werden die Triples in Listing 13 erstellt.

```
rdfash:Klassenlabel.AttributLabel-datatype
    sh:datatype xsd:float;
    sh:nodeKind sh:Literal;
    sh:message "The datatype is not a literal or it violates the xsd datatype.".
```

Listing 13: Muster primitive -datatype PropertyShape

Demnach muss ein Instanzwert ein Literal mit dem angegebenen XSD-Datentyp xsd:float sein.

Im Falle, dass das Prädikat rdfs:range vorhanden ist, handelt es sich bei dem Datentypen um eine Enumerationsklasse, welche durch den cims:stereotype http://iec.ch/TC57/NonStandard/UML#enumeration gekennzeichnet ist. Statt auf einen Datentyp zu überprüfen, wird hier über das Prädikat sh:in auf eine Liste von IRIs verwiesen, welche der Instanzwert annehmen darf. In diesem Fall werden die Statements sh:in (…), sh:nodeKind sh:IRI und sh:message mit entsprechender Fehlermeldung erstellt. Bei einer Enumerationsklasse cim:OrientationKind, mit den zugehörigen Enumerationswerten cim:OrientationKind.positive und cim:OrientationKind.negative, werden folglich die in Listing 14 dargestellten Triples erstellt.

```
rdfash:Klassenlabel.AttributLabel-datatype
    sh:in        (cim:OrientationKind.positive cim:OrientationKind.negative);
    sh:nodeKind sh:IRI;
    sh:message  "The datatype is not an IRI or its enumerated value is not part of the profile.".
```

Listing 14: Muster enum -datatype PropertyShape

Nach dem Listing 14 muss ein Instanzwert die IRI eines gültigen Enumerationswertes als Objekt haben. In diesem Fall entweder `cim:OrientationKind.positive` oder `cim:OrientationKind.negative`.

Assoziation zu PropertyShapes

Eine CIM-Assoziation ist dadurch gekennzeichnet, dass sie neben dem `rdf:type rdf:Property` Statement die Prädikate `cims:AssociationUsed` und `cims:inverseRoleName` aufweist. Deren Validität kann überprüft werden, indem der Typ und die Kardinalität überprüft werden. Anders als bei den Attributen, wo Datentypklassen beliebig oft genutzt werden dürfen, kann eingeschränkt werden, wie oft eine Klasse durch eine Assoziation referenziert werden darf. Ist dies der Fall und die referenzierte Klasse instanziierbar, kann zusätzlich die Kardinalität der Rückrichtung der Assoziation validiert werden. Um die Kardinalität einer Assoziation zu überprüfen, kann also ein `-cardinality` und ein `-inverseCardinality` PropertyShape erstellt werden. Das `-cardinality` PropertyShape wird wie oben beschrieben erstellt. Um zu validieren, dass die Assoziation auf eine Instanz der erlaubten Klassen zeigt, wird neben dem `-valueType` PropertyShape ein `-hasType` PropertyShape erstellt. Dieses zielt darauf ab, dass überprüft wird, ob der Pfad des `-valueType` PropertyShapes existiert. Auch wenn das Fehlen des `rdf:type` einer referenzierten Ressource nicht direkt ein Fehler ist, wird hier anstatt einer `sh:Violation` eine `sh:Warning` geworfen und erwähnt, dass eine Überprüfung des Datentyps nicht möglich ist.

Um das `-valueType` PropertyShape zu erstellen, muss die IRI der Assoziation, die der Klasse, auf welche die Assoziation zeigt, sowie alle instanziierbaren Unterklassen dieser ermittelt werden. Die IRI der Zielklasse kann über das Prädikat `rdfs:range` ausgelesen werden. Die direkt erbenden Klassen können darüber ermittelt werden, dass diese die Zielklasse mittels des Prädikats `rdfs:subClassOf` referenzieren. Da auch indirekt erbende Klassen, d.h. Klassen, welche von einer erbenden Klasse erben, als referenzierte Klassen erlaubt sind, ist es möglich, dass sich ein gesamter Vererbungsbaum aufspannt, welcher berücksichtigt werden muss. Der RDF-Architect erlaubt zum jetzigen Zeitpunkt zirkuläre Referenzen. Dies soll in der Implementierung berücksichtigt werden, um Endlosschleifen oder andere Fehler zu vermeiden. Für das PropertyShape werden neben den für alle PropertyShapes zu erstellenden Triples die Statements `sh:nodeKind sh:IRI` und `sh:in (…)` erstellt, wobei in der Liste alle instanziierbaren Unterklassen der Zielklasse sowie die Zielklasse selbst, stehen. Es wird außerdem das Prädikat `sh:path` erstellt, welches als Objekt eine Liste mit zwei Einträgen hat. Der erste ist die IRI der Assoziation und der zweite `rdf:type`. Eine Instanz mit der Assoziation muss also via dieser auf eine andere Instanzklasse zeigen, deren `rdf:type` in der `sh:in` Liste stehen. Ein `-valueType` PropertyShape besteht also aus den in Listing 15 beschriebenen Triples.

```
rdfash:Klassenlabel.AssoziationsLabel-valueType
    rdf:type          sh:PropertyShape;
    sh:name           "Klassenlabel.AssoziationsLabel-valueType";
    sh:description  "This constraint validates the value type of the association at the used direction."
    sh:message "One of the following occurs: 1) The value type is not an IRI; 2) The value type is not the right class.";
    sh:order          Die Order der Property(z.B. 2.0);
    sh:group          rdfash:ValueTypeGroup;
    sh:severity       sh:Violation .
    sh:in             ( [liste erlaubter Klassen] );
    sh:nodeKind       sh:IRI;
    sh:path           ( [associationIRI] rdf:type );
```

Listing 15: Muster -valueType PropertyShape

Die `sh:description`, die `sh:message`, sowie die `sh:group` sind über alle `-valueType` PropertyShapes konstant.

Um das `-hasType` PropertyShape zu erstellen, wird die IRI der Assoziation benötigt, welche bereits für das `-valueType` PropertyShape ermittelt wurde. Anders als bei der `-valueType` PropertyShape, die den Pfad `Assoziation rdf:type` validiert, fokussiert sich die `-hasType` PropertyShape ausschließlich darauf, dass die über die Assoziation referenzierte Ressource überhaupt ein `rdf:type` Triple besitzt. Hierfür wird eine verschachtelte NodeShape-Struktur verwendet.

Das `sh:node` Prädikat in SHACL ermöglicht es, dass alle Werte eines bestimmten Pfades gegen eine separate NodeShape validiert werden müssen. Im Kontext der `-hasType` PropertyShape bedeutet dies: Jede Ressource, die über den `sh:path` der Assoziation erreicht wird, muss die Constraints der referenzierten NodeShape erfüllen. Diese Verschachtelung ist notwendig, da die direkte PropertyShape nur den Pfad zur referenzierten Ressource definiert, aber nicht deren interne Struktur validieren kann.

Die referenzierte NodeShape wird als Blank Node implementiert und enthält ihrerseits eine PropertyShape, welche spezifisch das Vorhandensein des `rdf:type` Triples validiert. Diese innere PropertyShape erhält die Constraints `sh:path rdf:type` und `sh:minCount 1`. Dadurch wird für jede über die Assoziation referenzierte Ressource überprüft, ob sie mindestens ein `rdf:type` Triple besitzt, unabhängig davon, welcher konkrete Typ angegeben ist.

Für das PropertyShape werden neben den für alle PropertyShapes zu erstellenden Triples die Statements `sh:path`, welches direkt auf die Assoziations-IRI zeigt, und `sh:node` erstellt. Ein `-hasType` PropertyShape besteht dem zur Folge aus den in Listing 16 beschriebenen Triples.

```
rdfash:Klassenlabel.AssoziationsLabel-hasType
    rdf:type sh:PropertyShape;
    sh:name          "Klassenlabel.AssoziationsLabel-hasType";
    sh:description     "This constraint validates that an association points to a node with an rdf:type.";
    sh:message "The referenced node is missing an rdf:type, therefore a type validation is not possible.";
    sh:order          [Die Order der Property];
    sh:group          rdfash:HasTypeGroup;
    sh:severity       sh:Warning;
    sh:path           [associationIRI];
    sh:node [
        rdf:type      sh:NodeShape;
        sh:property [
                sh:path rdf:type;
                sh:minCount 1;
        ];
    ].
```

Listing 16: Muster -hasType PropertyShape

Die `sh:description`, die `sh:message` sowie die `sh:group` sind über alle `-hasType` PropertyShapes konstant. Diese Validierung ergänzt die `-valueType` PropertyShape optimal, da sie sicherstellt, dass der für die Typvalidierung notwendige `rdf:type` Pfad überhaupt existiert, bevor die spezifische Klassenvalidierung erfolgt.

Zuletzt kann noch die inverse Kardinalität einer Assoziation überprüft werden. Wie in der Analyse beschrieben, wird diese nicht erstellt, wenn die inverse Kardinalität unbestimmt oder die von der Assoziation referenzierte Klasse nicht instanziierbar ist. Ansonsten muss hierfür die `cims:multiplicity` der inversen Assoziation, welche über `cims:inverseRoleName` erreichbar ist, ausgelesen werden. Hier werden analog zum normalen `-cardinality` PropertyShape die `sh:minCount` und `sh:maxCount` Constraints erstellt. Auch die möglichen Werte unter `sh:message` und `sh:nodeKind` sind identisch. Nur die `sh:description`, `sh:propertyGroup` und die `sh:path` Statements unterscheiden sich. Die `sh:description` ist immer `This constraint validates the cardinality of the property (association).`. Die `sh:propertyGroup` ist die `rdfash:inverseCardinalityGroup`. Der `sh:path` zeigt, wie in der Analyse bereits erläutert, zu einem Blank Node, welcher mittels des Statements `sh:inversePath` zur IRI der Assoziation zeigt. Eine `-inverseCardinality` PropertyShape für eine Assoziation mit der Multiplizität zwei bis drei wird somit durch die in Listing 17 beschriebenen Triples dargestellt.

```
rdfash:KlassenName.associationLabel-inverseCardinality
    rdf:type          sh:PropertyShape;
    sh:description "This constraint validates the cardinality of the property (association)";
    sh:group          rdfash:inverseCardinalityGroup;
    sh:minCount       2;
    sh:maxCount       3;
    sh:message        "Cardinality violation 2..3 (association).";
    sh:name           "KlassenName.associationLabel-inverseCardinality";
    sh:order          Die Order der Property(z.B. 1.0);
    sh:path           [ sh:inversePath associationIRI];
    sh:severity       sh:Violation .
```

Listing 17: Muster -inverseCardinality PropertyShape

4.2.2 CIM-Class zu NodeShape

Nach der Generierung aller PropertyShapes für eine Klasse generiert wurden wird das zu der Klasse zugehörige NodeShape erstellt. Im Falle, dass kein PropertyShape erstellt wurde, wird die Erstellung des NodeShapes abgebrochen und die nächste Klasse betrachtet. Ansonsten wird zuerst die IRI erstellt. Diese besteht aus dem `rdfs:label` der CIM-Klasse und dem SHACL-Präfix. Für das NodeShape werden dann die in Listing 18 beschriebenen Triples erstellt.

```
rdfash:classLabel
    rdf:type        sh:NodeShape;
    sh:property     PropertyShapeIRI;
    sh:targetClass  classIRI .
```

Listing 18: Muster NodeShape

Für jedes PropertyShape wird ein zusätzliches `sh:property` Statement mit neuem Objekt erstellt. Im Fall, dass die NodeShapes closed sein sollen, d.h. dass von PropertyShapes nicht validierte Triples nicht existieren dürfen, werden zusätzlich die beiden in Listing 19 beschriebenen Triples erstellt.

```
rdfash:classLabel
    sh:closed             true;
    sh:ignoredProperties ( rdf:type );
```

Listing 19: Muster closed NodeShape Erweiterung

`sh:ignoredProperties` definiert hierbei, dass `rdf:type` Triples ausgenommen sind. Das wird getan, da diese notwendig sind um Instanzen zu einer Klasse zuzuordnen.

4.2.3 PropertyGroups

Zur organisatorischen Strukturierung der generierten PropertyShapes werden PropertyGroups verwendet, die thematisch verwandte PropertyShapes gruppieren. Für jede PropertyGroup, welche von einem PropertyShape über `sh:propertyGroup` referenziert wird, wird eine PropertyGroup mit den in Listing 20 beschriebenen Triples erstellt.

```
rdfash:GroupName
    rdf:type    sh:PropertyGroup;
    rdfs:label  "GroupName";
    sh:order    1.0 .
```

Listing 20: Muster PropertyGroup

Die `sh:order` ist der Index der PropertyGroup in einer nach dem `rdfs:label` sortierten Liste.

Mit diesem Konzept ist eine automatische Generierung der strukturell ableitbaren SHACL-Validierungsregeln aus CIM/XML-Ontologien möglich. Die definierten Regeln bilden die Basis für die nachfolgend in Kapitel 5 beschriebene technische Implementierung im RDF-Architect.

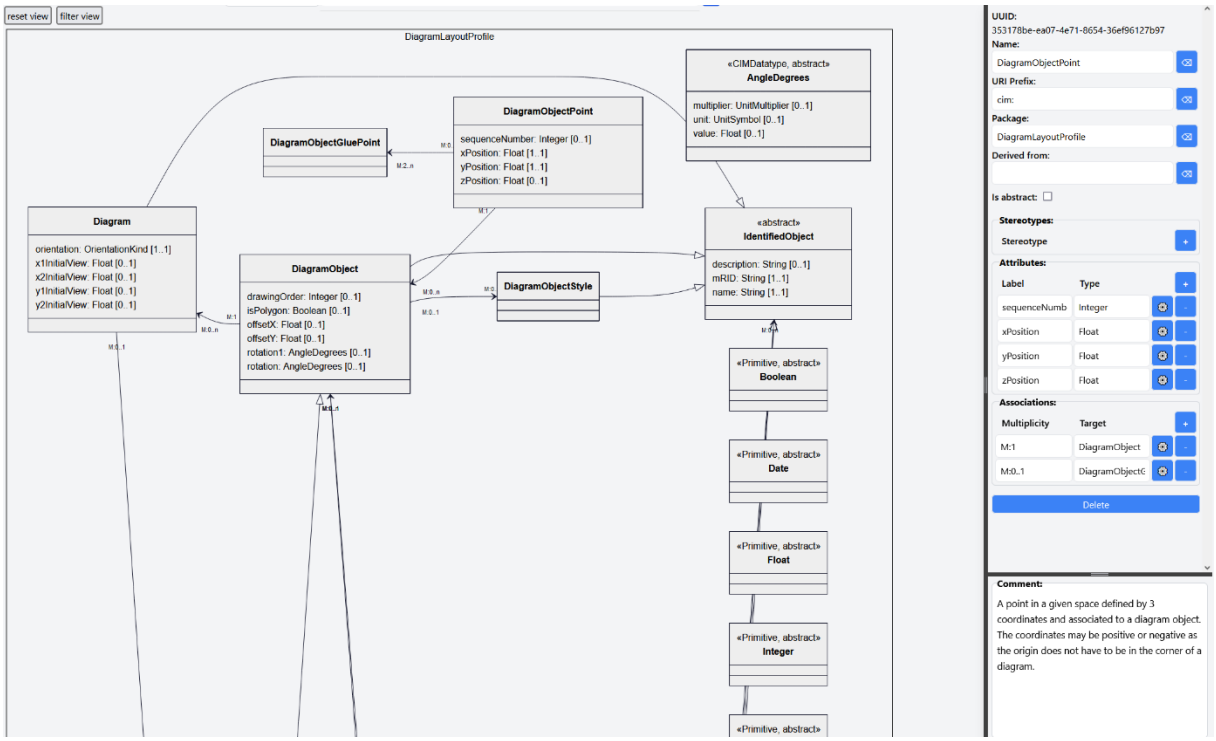
4.3 Visualisierung von SHACL im RDF-Architect

Generierte und importierte SHACL-Regeln sollen im RDF-Architect auch angezeigt werden. Im Folgenden wird das im Rahmen der Arbeit umgesetzte Konzept beschrieben, welches dies realisiert. Die SHACL-Regeln können dadurch sowohl im Ganzen als auch kontextspezifisch angezeigt werden.

Eine Zuordnung eines NodeShape zu einer Klasse ist über das `sh:targetClass` Prädikat möglich. Eine zu einem PropertyShape zugehörige Property zu finden, kann aufwendig bis unmöglich sein. Ein durch den Nutzer erstelltes PropertyShape könnte zum Beispiel den Namen der Klasse auf eine bestimmte Zeichenlänge prüfen. In diesem Fall gibt es keine Property, welche dem PropertyShape zugeordnet werden könnte. Ein anderer Fall wäre eine SPARQL-PropertyShape. Diese kann beliebig komplex sein, wobei eine eindeutige Zuordnung zu einer Property nicht zwingenderweise möglich ist. Deshalb ist die technisch umgesetzte Anforderung an die Anzeige, dass zumindest die generierten PropertyShapes in Relation zu einer Property angezeigt werden können. Importierte PropertyShapes werden im erweiterten RDF-Architect in Relation zu Properties angezeigt, wenn diese über das für generierte PropertyShapes erstellte Zuordnungsmuster zu einer Property zugeordnet werden können. Andernfalls wird versucht, diese im Kontext des NodeShapes beziehungsweise der Klasse anzuzeigen.

Um die Möglichkeit zu bieten, auch verwaiste Regeln, d.h. solche wofür kein Zusammenhang zu dem CIM-Modell gefunden werden kann, anzuzeigen, wurde eine Ansicht realisiert, die es erlaubt, die gesamte importierte SHACL-Datei anzuzeigen. Es wurden eine generelle sowie zwei weitere Ansichten erstellt, wobei sich die letzteren auf den Kontext einer Klasse bzw. den Kontext einer einzelnen Property beschränken.

Die Library mermaid.js, welche im RDF-Architect für das Visualisieren und die Anordnung des UML-Klassendiagramms genutzt wird, hat wesentliche Einschränkungen und lässt dem Programmierer nur wenig Freiheiten, um ein interaktives Design zu erstellen. Es ist lediglich möglich, durch einen Mausklick auf eine Klasse, einen Funktionsaufruf zu triggern. Dies wird im RDF-Architect genutzt, um in einem Rightbar, d.h. in einem rechts angedockten Fenster eine, wie Listing 21 dargestellte Detailansicht, auch class-Editor genannt, zu öffnen.



Listing 21: class-Editor

Das Klicken individueller Eigenschaften sowie das Anzeigen eines Pop-ups in Relation zu einer bestimmten Property der Klasse wird von mermaid.js nicht unterstützt. Da dies jedoch über den class-Editor möglich ist, wurde die SHACL-Erweiterung dort eingepflegt.

Im Folgenden werden die Konzepte der zwei realisierten Ansichten dargestellt: Eine Ansicht, welche Regeln zu einer spezifischen Property anzeigt und eine weitere, die alle Regeln im Kontext einer Klasse anzeigt. Es können sowohl importierte als auch generierte Regeln angezeigt werden.

Importierte Regeln sind durch den Nutzer bearbeitbar. Dies erfolgt unter der Annahme, dass dieser nur korrekte Eingaben macht. Das Editieren des CIM-Modells hat keine Auswirkungen auf die importierten Regeln. Das bedeutet, dass aus einer Namensänderung einer Klasse folgt, dass ein direkter Zugriff auf die SHACL-Regeln über die Klasse mit jetzt geändertem Namen nicht mehr möglich ist. Das automatische Anpassen der importierten Regeln kann sehr komplex werden und war nicht Teil des Scopes dieser Arbeit.

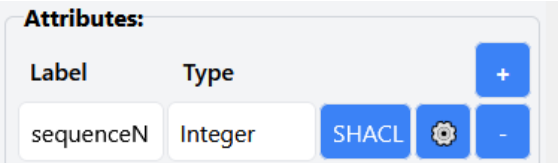
Generell werden alle SHACL-Regeln als Text im Turtle Format angezeigt. Dies bietet sich für die Endnutzerinteraktion an, da es ein gängiges und für Menschen einfach lesbares Format ist. Um die Lesbarkeit weiter zu unterstützen, wurde ein farbiges Syntax-Highlighting für die Turtle-Darstellung implementiert.

4.3.1 Property-spezifische Ansicht

Um mehr Details über ein Attribut als nur das Label und den Typ betrachten zu können, wurde im RDF-Architect eine Detailansicht für Attribute realisiert. Sie lässt sich über das Klicken des Zahnrads neben der jeweiligen Property öffnen. Es könnte sich sowohl logisch als auch funktionell anbieten, die SHACL-Ansicht in diese Detailansicht einzubinden. Da diese

Detailansicht jedoch noch nicht implementiert ist, wird vorerst ein eigener Pop-up erstellt. Ob dieser später in die eigentliche Attribut-Detailansicht eingebunden wird, kann im weiteren Verlauf des RDF-Architect Projekts entschieden werden.

Es wurde daher ein Button eingefügt, welcher ein Pop-up öffnet, welches Informationen über die zu dieser Property zugehörigen SHACL-Regeln anzeigt. Der Button sieht wie in Abbildung Listing 22 dargestellt aus.



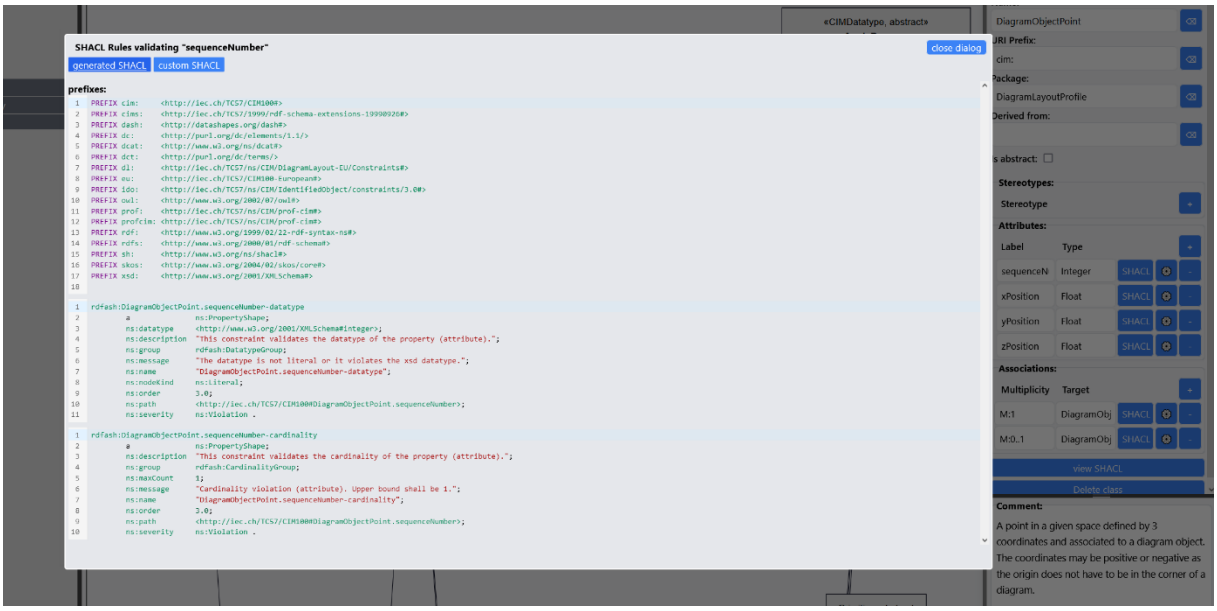
Listing 22: Propertydarstellung im classeditor des RDF-Architect

Das durch einen Klick geöffnete Pop-up enthält die SHACL-Regeln, welche zu der Property zugeordnet werden können. Dieser Pop-up enthält die PropertyShapes, welche in Zusammenhang mit der jeweiligen Property stehen. Diese Zuordnung ist mindestens für die eigen-generierten PropertyShapes möglich. Das bedeutet, dass eine PropertyShape als zugehörig zu einer Property gilt, wenn die IRI der Property das Objekt des Prädikats `sh:path` der PropertyShape ist. Eine andere Option ist, dass das Objekt eine Liste an Pfaden ist. Enthält diese Liste die IRI der Property, wird die PropertyShape auch als zu der Property zugehörig angesehen.

In dem Fall, dass das Objekt ein Blank Node ist, welches über `sh:InversePath` zu der IRI der Property zeigt, wird angenommen, dass es sich bei der Property um eine Assoziation handelt. Dieses PropertyShape wird der inversen Richtung der Assoziation zugeordnet.

Neben den zugeordneten Regeln werden außerdem die Präfixe zur Vollständigkeit der Informationen angezeigt. Da diese jedoch nicht essenziell sind und viel Platz einnehmen können ohne großen Mehrwert zu bieten, können diese versteckt werden, um die Übersichtlichkeit über die eigentlichen PropertyShapes zu bewahren.

Da sowohl die Informationen der generierten als auch importierten SHACL-Regeln angezeigt werden sollen, existiert für beide jeweils ein unabhängiger Tab. Eine Ansicht der Regeln sieht aus wie in Listing 23 dargestellt.



Listing 23:Propertyspezifische Ansicht für generiertes SHACL

Bei einem Klick auf `prefixes`: werden die Präfixe ein- oder ausgeblendet. Beim Auswählen von `custom SHACL` bleibt die Ansicht, bis auf den Inhalt der angezeigten SHACL-Regeln, zunächst identisch. Der einzige Unterschied ist, dass generierte PropertyShapes read-only sind, während Importierte bearbeitbar sind. Werden die PropertyShapes editiert, erscheint ein Button, welcher das Speichern dieser erlaubt. Auch hier wird vom Nutzer erwartet, dass er korrekte Eingaben durchführt. Die Ansicht sieht aus wie in Listing 24 dargestellt.

Listing 24: Propertyspezifische Ansicht für custom SHACL

4.3.2 Klassenspezifische Ansicht

Die zweite realisierte Ansicht beinhaltet den gesamten Kontext einer Klasse, d.h. alle SHACL-Regeln, die auf eine Instanz einer Klasse angewendet werden. Hier erfolgt die Zuordnung dadurch, dass zuerst die zu einer Klasse zugehörigen NodeShapes gesucht werden. Zum Kontext der Klasse gehören alle von deren NodeShapes über `sh:propertyShape` referenzierten PropertyShapes. Es werden also die NodeShapes, die PropertyShapes und zur Vollständigkeit auch wieder die Präfixe angezeigt. Zur besseren Übersichtlichkeit sind diese erneut ausblendbar. Unter den Präfixen werden die NodeShapes angezeigt. Diese erhalten jeweils einen eigenen Abschnitt und können ebenfalls individuell ausgeblendet werden. Danach folgen die PropertyShapes. Diese sind unterteilt in Abschnitte, welche diese den jeweiligen Properties zuordnen. Diese Abschnitte sind nach der `sh:order` der jeweils ersten PropertyShape des Abschnitts sortiert. Falls ein PropertyShape keiner Property zugeordnet werden kann, wird diese in einem eigenen Abschnitt angezeigt. Falls die Klasse zu validierende Properties von einer anderen erbt, werden diese in einem separaten Abschnitt `derived PropertyShapes` angezeigt.

SHACL shapes

generated SHACL | custom SHACL

prefixes:

NodeShapes:

Diagram:

```
1  rdfash:Diagram  rdf:type      sh:NodeShape;
2          sh:closed             true;
3          sh:ignoredProperties  ( rdf:type );
4          sh:property           rdfash:IdentifiedObject.mRID-cardinality , rdfash:Diagram.y1InitialView-cardinality , rdfash:IdentifiedObject.description-datatype , rdfash:Dia
5          sh:targetClass        cim:Diagram .
```

PropertyShapes:

Diagram.DiagramStyle:

```
1  rdfash:Diagram.DiagramStyle-cardinality
2          rdf:type        sh:PropertyShape;
3          sh:description  "This constraint validates the cardinality of the property (association).";
4          sh:group        rdfash:CardinalityGroup;
5          sh:maxCount     1;
6          sh:message      "Cardinality violation (association). Upper bound shall be 1.";
7          sh:name         "Diagram.DiagramStyle-cardinality";
8          sh:order        1.0;
9          sh:path         cim:Diagram.DiagramStyle;
10         sh:severity     sh:Violation .
```

```
1  rdfash:Diagram.DiagramStyle-valueType
2          rdf:type        sh:PropertyShape;
3          sh:description  "This constraint validates the value type of the association at the used direction.";
4          sh:group        rdfash:ValueTypeGroup;
5          sh:in           ( cim:DiagramStyle );
6          sh:message      "One of the following occurs: 1) The value type is not IRI; 2) The value type is not the right class.";
7          sh:name         "Diagram.DiagramStyle-valueType";
8          sh:nodeKind     sh:IRI;
9          sh:order        1.0;
10         sh:path         ( cim:Diagram.DiagramStyle rdf:type );
11         sh:severity     sh:Violation .
```

Diagram.orientation:

Diagram.x1InitialView:

Diagram.x2InitialView:

Diagram.y1InitialView:

Diagram.y2InitialView:

derived PropertyShapes:

IdentifiedObject.description:

IdentifiedObject.mRID:

IdentifiedObject.name:

Listing 25: Klassenspezifische Ansicht für generiertes SHACL

Auch hier gibt es wieder die Möglichkeit, die importierten SHACL-Regeln zu bearbeiten. Nach demselben Prinzip wie der Property-spezifischen Ansicht erscheint hier bei Anpassung der

Regeln ein Button zum Speichern dieser. Geerbte Property Shapes sind nur in der Ansicht der Klasse, zu der sie gehören, anpassbar. Listing 26 zeigt eine entsprechende Darstellung.

SHACL shapes

generated SHACL | custom SHACL

Save changes

prefixes:

NodeShapes:

DiagramObjectGluePoint:

```
1  dli:DiagramObjectGluePoint
2          rdf:type        sh:NodeShape;
3          sh:property     dli:DiagramObjectGluePoint.DiagramObjectPoints-cardinality;
4          sh:targetClass  cim:DiagramObjectGluePoint .
```

PropertyShapes:

DiagramObjectGluePoint.DiagramObjectPoints:

```
1  dli:DiagramObjectGluePoint.DiagramObjectPoints-cardinality
2          rdf:type        sh:PropertyShape;
3          sh:description  "This is an edited  description of this PropertyShape";
4          sh:group        dli:InverseAssociationsCardinality;
5          sh:message      "Wrong number of associated instances.";
6          sh:minCount     2;
7          sh:name         "DiagramObjectGluePoint.DiagramObjectPoints-cardinality";
8          sh:order        1;
9          sh:path         [ sh:inversePath  cim:DiagramObjectPoint.DiagramObjectGluePoint ];
10         sh:severity     sh:Violation .
```

no derived propertyShapes found

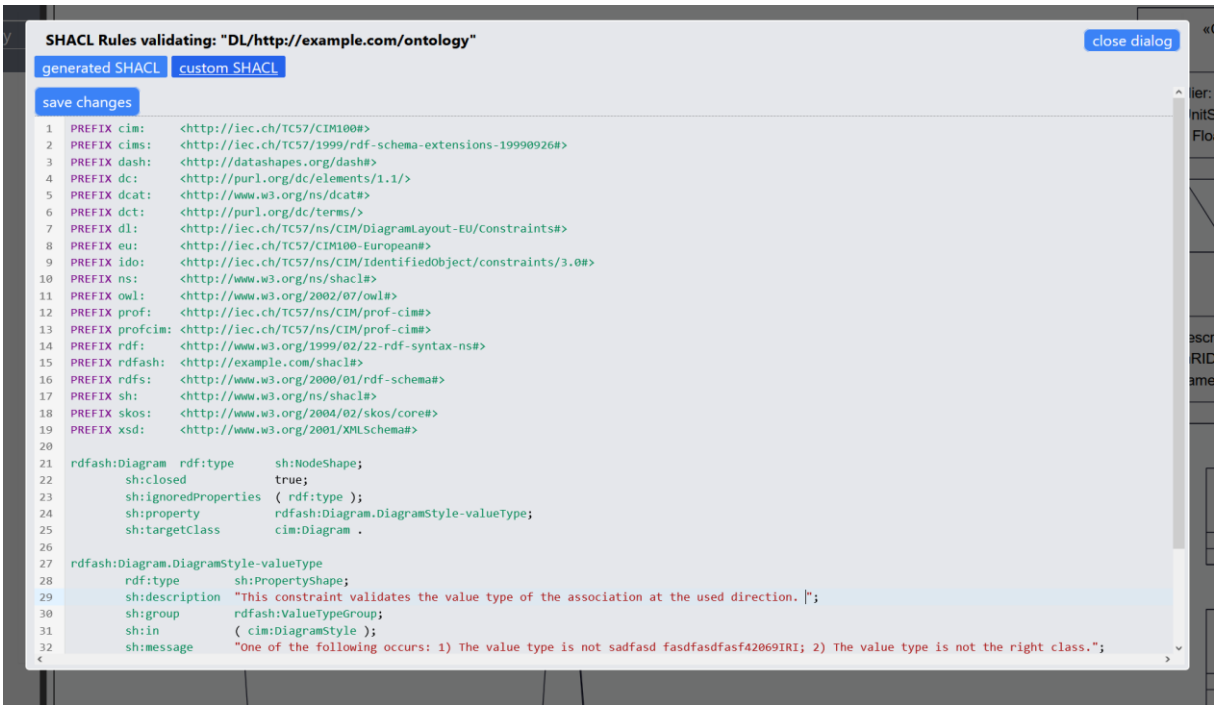
add new custom shapes:

Listing 26: Klassenspezifische Ansicht für custom SHACL

Unter der Ansicht der PropertyShapes gibt es die Möglichkeit, neue SHACL-Regeln manuell hinzuzufügen. Durch einen Klick auf dieses Label, welches gleichzeitig als Button fungiert, wird ein Textfeld geöffnet, das dem Nutzer erlaubt, eigene Shapes hinzuzufügen.

4.3.3 Vollständige Ansicht

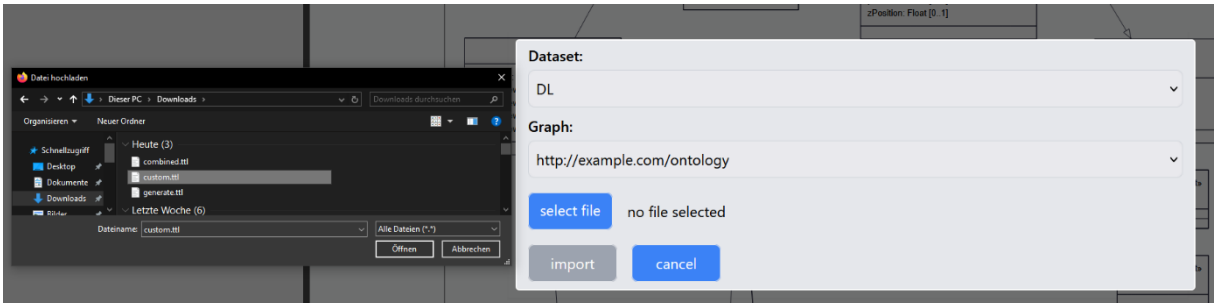
Eine weitere Ansichtsmöglichkeit zeigt die gesamten SHACL-Regeln an. Auch diese ist aufgeteilt in einen Tab für die generierten und einen für importierte SHACL-Regeln, wobei ebenfalls nur letztere bearbeitbar sind. Diese in Listing 27 dargestellte Ansicht lässt sich über die Menüleiste öffnen und zeigt die SHACL-Regeln für den aktuell ausgewählten Graphen an.



Listing 27: Vollständige Ansicht für custom SHACL

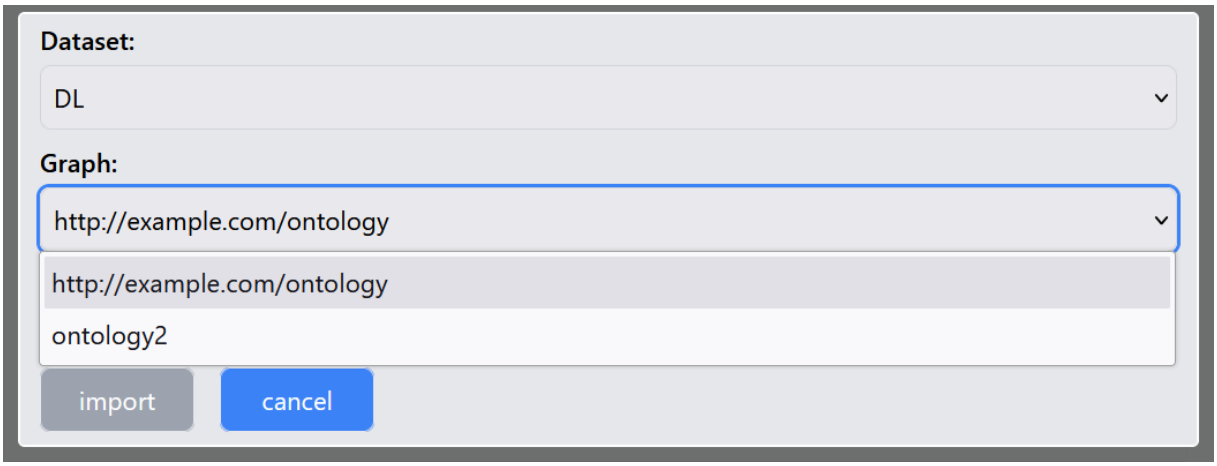
4.3.4 Import-Dialog

Um eigens erstellte Shapes zu importieren, gibt es einen Dialog zum Importieren der SHACL-Regeln. Dieser orientiert sich an bereits vorhandenen Dialogen. Es wird das Layout sowie die Dateiauswahl aus dem Ontologie-Upload übernommen. Dieser öffnet den Dateiexplorer und erlaubt die Auswahl über das Dateisystem des Betriebssystems.



Listing 28: Importdialog für custom SHACL

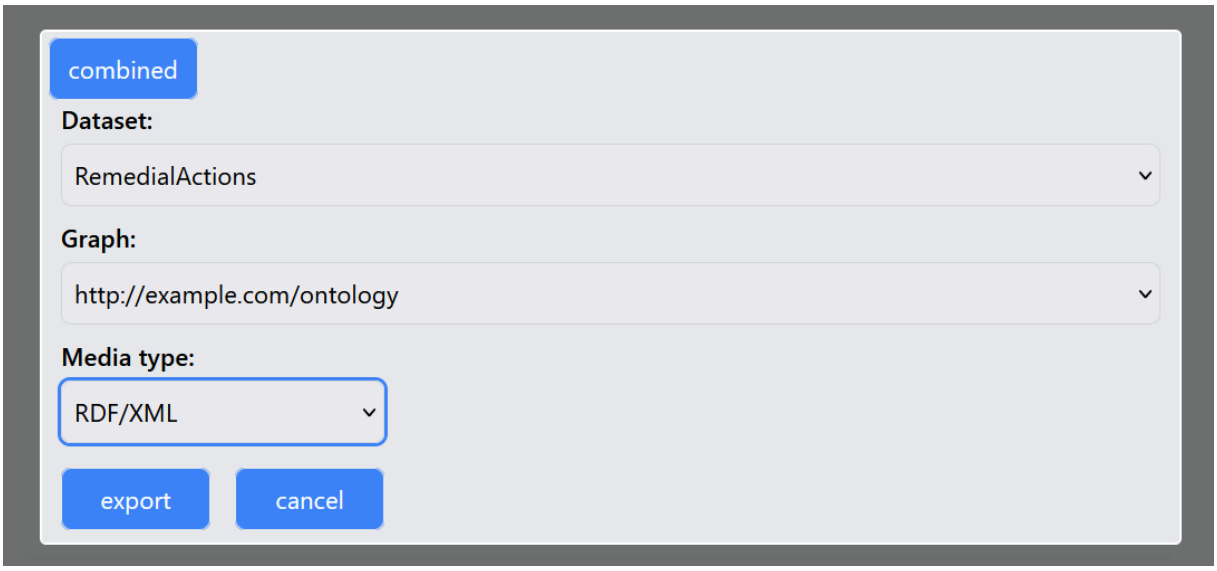
Die Auswahl eines Datensatzes und Graphen wurde über ein Drop-down-Menu wie in Listing 29 dargestellt, gelöst.



Listing 29: Importdialog für custom SHACL dropdown

4.3.5 Export Dialog

Um generierte und eigen-importierte, gegebenenfalls angepasste SHACL Regeln zu exportieren, wurde ein Dialog für den Export realisiert. Auch dieser orientiert sich an dem bereits vorhandenen Dialog zum Exportieren von Ontologien. Der Unterschied ist, dass es wie in Listing 30 einen zusätzlichen Button gibt, welcher durch einen Klick wechselt, was exportiert werden soll.



Listing 30: Exportdialog für SHACL

Möglich ist der Export der generierten oder importierten SHACL-Regeln sowie deren Zusammenführung dieser. Dabei wird eine Vereinigung aus beiden Graphen erstellt und geprüft, ob eine Ressource mit einer gewissen IRI in beiden Graphen existiert. Ist dies der Fall, wird nur die der importierten SHACL-Regeln übernommen, ansonsten die Ressource. Auch hier geschieht das Abspeichern wie beim Import über das Dateisystem des Betriebssystems.

4.4 Frontend-Backend-Schnittstelle

Die Verbindung zwischen Frontend und Backend erfolgt im RDF-Architect über eine RESTful API, an der sich die neu implementierten Routen orientieren. Das API-Design ist ressourcenorientiert und strukturiert die Endpunkte hierarchisch.

Endpunkte, welche sich auf ein spezifisches UML-Diagramm beziehen, identifizieren dieses immer über den Pfad `/api/datasets/{datasetName}/graphs/{graphURI}`. Datasets fungieren als übergeordnete Container für Graphen, die die UML-Diagramme beinhalten.

Soll beispielsweise eine Klasse aus einem UML-Diagramm abgefragt werden, so wird der Pfad um das Suffix `/classes/{classUUID}` erweitert. Um einen Zugang zu SHACL über die API zu gewähren, werden Pfade um das Suffix `/shacl` erweitert. Wenn benötigt, wird noch zusätzlich über ein weiteres Suffix `/generated` oder `/custom` spezifiziert, ob es sich um generiertes oder custom shacl handelt.

5 Details zur Realisierung

Im Rahmen dieses Kapitels werden die zentralen Implementierungsaspekte dokumentiert. Sämtliche erwähnten Methoden sind, falls nicht explizit anders beschrieben, eigens für diese Arbeit implementiert.

5.1 Auswahl und Integration der Technologien

Die Technologiewahl für die SHACL-Erweiterung orientierte sich primär an der bestehenden RDF-Architect-Architektur, um Kompatibilität, Einheitlichkeit und Wartbarkeit des erweiterten Systems zu gewährleisten. Neue Bibliotheken wurden nur dann integriert, wenn sie wesentliche Funktionalitäten bereitstellten, die im "Status Quo" Tech-Stack vor dem Start der Arbeiten nicht verfügbar oder unverhältnismäßig aufwendig zu implementieren waren.

5.1.1 Backend Technologien

Das Backend des RDF-Architects basiert auf Java mit dem Spring Boot Framework. Diese Kombination bietet eine bewährte, enterprise-taugliche Grundlage mit umfangreichen Entwicklungstools und einer aktiven Community. Spring Boot vereinfacht dabei die Konfiguration und das Deployment erheblich und integriert sich nahtlos mit den benötigten Bibliotheken.

Für die RDF-Datenverarbeitung im existierenden System wird das Apache Jena Framework verwendet, auf dem die interne RDF-basierte Logik bereits fundamental aufbaut. Besonders relevant ist Jenas umfassende RDF-Unterstützung, da die Datenhaltung im RDF-Architect vollständig in RDF-Graphen erfolgt. Jena stellt alle erforderlichen APIs für RDF-Manipulation, SPARQL-Queries und die Arbeit mit verschiedenen RDF-Serialisierungsformaten (Turtle, RDF/XML, JSON-LD, etc.) bereit. Die Bibliothek bietet performante Verarbeitung von RDF-Graphen. Zusätzlich ermöglicht Jenas SHACL-Modul die Validierung der generierten Regeln gegen Testdaten.

Das Test-Framework für den Test der Erweiterungen basiert auf JUnit 5, welches bereits standardmäßig im RDF-Architect verwendet wird. JUnit 5 bietet moderne Testing-Features wie parametrisierte Tests, dynamische Tests und erweiterte Assertion-Möglichkeiten. Ergänzend wird AssertJ eingesetzt, das durch seine Fluent-API die Lesbarkeit und Wartbarkeit der Tests erheblich verbessert. AssertJ ist bereits im RDF-Architect etabliert und bietet Quality-of-Life-Verbesserungen wie aussagekräftige Fehlermeldungen und eine intuitive Syntax für komplexe Assertions.

Lombok wird sowohl im existierenden System als auch in der SHACL Erweiterung für die Reduzierung von Boilerplate-Code verwendet. Die hierdurch bereitgestellten Annotationen wie @Data eliminieren beispielsweise repetitive Getter/Setter-Implementierungen, die in der SHACL-Generierung häufig verwendet werden.

5.1.2 Frontend Technologien

Das Frontend des RDF-Architects basiert auf SvelteKit mit Svelte 5, einem reaktiven JavaScript-Framework. Da die SHACL-Erweiterung direkt in den RDF-Architect eingebunden werden sollte, war dieses Framework Voraussetzung für die Erweiterung.

Svelte 5 bringt verschiedene Features wie Reaktivität mit sich. Die Komponentenarchitektur von Svelte ermöglichte eine modulare Implementierung der verschiedenen SHACL-Ansichten, wie der Property-spezifischen, der Klassenspezifischen und der vollständigen Ansicht.

Für das Styling wird Tailwind CSS verwendet, ein Utility-First CSS-Framework, das im gesamten RDF-Architect konsistent eingesetzt wird. Tailwind ermöglicht eine schnelle Entwicklung durch vorgefertigte Utility-Classes und erleichtert dadurch die Einhaltung eines einheitlichen Design-Systems.

Für die Darstellung der SHACL-Regeln im Turtle-Format war die Auswahl einer geeigneten Syntax-Highlighting-Bibliothek entscheidend. Zunächst wurde Prism.js evaluiert, da es eine leichtgewichtige Lösung für Syntax-Highlighting bietet und bereits gute Turtle-Unterstützung hat. Prism.js ist primär für read-only Code-Darstellungen konzipiert, was bedeutet, dass ein Syntax-Highlighting während des Editierens der SHACL-Regeln nur über das Übereinanderlegen und gleichzeitige Synchronisieren zwischen einem separaten Editor-Feld und dem Highlighting-Panel möglich gewesen wäre. Diese Lösung hätte eine bidirektionale Synchronisation zwischen zwei verschiedenen DOM-Elementen erfordert, was fehleranfällig und schwer zu warten gewesen wäre, weshalb Prism.js als suboptimale Lösung eingestuft wurde.

Stattdessen wurde CodeMirror 6 gewählt, eine vollständige Code-Editor-Bibliothek, die Syntax-Highlighting und Editier-Funktionalitäten in einem einzigen, gut integrierten System bietet. CodeMirror 6 ist deutlich umfangreicher und nicht so leichtgewichtig wie Prism.js, bietet jedoch die erforderliche nahtlose Integration von Anzeige und Bearbeitung des Turtle-Codes. Die Bibliothek ist modular aufgebaut und ermöglicht die gezielte Integration der benötigten Features. Zusätzlich bietet die codemirror-lang-turtle Extension spezifisches Highlighting für das Turtle-Format mit Unterstützung für Präfixe, IRIs und Literale. CodeMirror bietet zusätzlich Features wie Syntax-Validierung, Auto-Completion und Bracket-Matching, die die Benutzererfahrung beim Bearbeiten von SHACL-Regeln erheblich verbessern.

Die Kommunikation zwischen Frontend und Backend erfolgt über die native Fetch API des Browsers. Diese bietet ausreichende Funktionalität für die RESTful API-Calls und ist bereits im RDF-Architect etabliert. Zusätzliche HTTP-Client-Bibliotheken wie Axios wurden bewusst nicht integriert, um die Bundle-Größe zu minimieren und die Abhängigkeiten gering zu halten.

5.2 Umsetzung der Generierung der SHACL-Regeln

Die Klasse SHACLFromCIMGenerator stellt zwei zentrale Methoden für das Generieren von SHACL-Regeln zum UML Diagramm zur Verfügung. Nach außen bietet die Klasse einmal die Methode `generate()`, welche ein Apache Jena Model zurückgibt, das die generierten SHACL-Regeln enthält. Die zweite public Methode ist `generateForClassOnly(UUID classUUID)`, welche die generierten SHACL-Regeln auf den Kontext einer Klasse einschränkt. Dieser zweite Endpunkt wird angeboten, um vor allem bei umfangreicheren CIM-Modellen performant zu bleiben, falls nur ein Teil der Regeln abgefragt wird.

5.2.1 Generierungsablauf

Zuerst wird die Klasse initialisiert, wobei das Ontologie-Modell, das SHACL-Präfix und ein Boolean, ob die generierten NodeShapes closed oder nicht closed sein sollen, übergeben

werden. Diese Entscheidung kann zum aktuellen Zeitpunkt zwar nicht vom Nutzer getroffen werden. Sie wurde jedoch bereits in den Code des Backends eingebunden, um zukünftig gegebenenfalls diese Möglichkeit zu bieten.

Nach dem Aufruf der `generate()`-Methode wird über CIM-Klassen iteriert. Dabei werden solche herausgefiltert, welche nicht instanziierbar sind oder keine validierbaren Eigenschaften haben.

Für jede qualifizierte Klasse werden anschließend alle zugehörigen Properties ermittelt. Dieser Prozess umfasst sowohl die Properties, die eine Klasse direkt besitzt, als auch solche, die sie über Vererbung erhält. Die Implementierung dieser Funktionalität erfolgt durch die Methode `listAllProperties(Resource classResource)` der Klasse `CIMClassUtils`, die als Parameter die Jena-Ressource der betrachteten Klasse erhält. Eine weitere Hilfsmethode derselben Klasse listet zuerst alle Superklassen auf.

Die Behandlung von Vererbungshierarchien kann im RDF-Architect problematisch sein, da dieser zirkuläre Referenzen durch die `rdfs:subClassOf`-Beziehung erlaubt, die zu Endlosschleifen führen können. Um dies zu vermeiden wurde für das Finden der Superklassen ein Set-Queue-Mechanismus genutzt. Zunächst werden alle direkten Superklassen sowohl einem Set als auch einer Queue hinzugefügt. Die Queue wird dann systematisch abgearbeitet, wobei für jede enthaltene Klasse erneut alle Superklassen ermittelt und zur weiteren Verarbeitung hinzugefügt werden. Das Set fungiert dabei als Schutz vor Duplikaten. Ist eine Klasse bereits enthalten, wird sie nicht erneut der Queue hinzugefügt und verhindert somit Endlosschleifen bei zirkulären Referenzen. Der Prozess wird fortgesetzt bis die Queue vollständig abgearbeitet ist und das Set alle eindeutigen Superklassen enthält. Nach dem Auffinden aller Superklassen werden deren Eigenschaften aufgelistet und zurückgegeben.

Generierung PropertyShapes

Über diese Liste von Properties wird dann iteriert, wobei für jede Property die zugehörigen Propertyshapes generiert werden. Um die Erweiterbarkeit des Codes zu verbessern, wurde nach dem Prinzip der Separation of Concerns das Erstellen der PropertyShapes in eigenen Klassen umgesetzt. Die Klasse SHACLFromCIMGenerator hat ein Feld, welches eine List von PropertyShapeFromCIMPropertyGenerator-Klassen enthält. Dies ist ein Interface, das die Methoden aus Listing 31 definiert.

```
PropertyShapeFromCIMPropertyGenerator setOntologyModel(Model ontology);
PropertyShapeFromCIMPropertyGenerator setShaclModel(Model shacl);
PropertyShapeFromCIMPropertyGenerator setShaclPrefix(PrefixEntry shaclPrefix);
Resource createPropertyShape(Resource property);
```

Listing 31: Interfacemethoden PropertyShapeFromCIMPropertyGenerator

Die ersten drei Methoden setzen nur den benötigten Kontext und lassen sich als Fluid Interface vorbereitend für die Methode `createPropertyShape()` aneinanderreihen. Diese hat letztendlich die Funktion, aus der gegebenen CIM-Property eine PropertyShape zu erstellen. Diese Architektur folgt dem Strategy Pattern, bei dem verschiedene Algorithmen,

wie hier PropertyShape-Generierungsstrategien, über ein gemeinsames Interface austauschbar implementiert werden.

Das Strategy Pattern ist ein Entwurfsmuster, das es ermöglicht, eine Familie von Algorithmen zu definieren, sie zu kapseln und austauschbar zu machen. In der vorliegenden Implementierung fungiert die `SHACLFromCIMGenerator`-Klasse als Context, das `PropertyShapeFromCIMPropertyGenerator`-Interface als Strategy-Schnittstelle und die verschiedenen Converter-Klassen für `-cardinality`, `-valueType`, `-hasType` und weitere als konkrete Strategy-Implementierungen. Der Vorteil dieses Patterns liegt in der hohen Erweiterbarkeit. Neue PropertyShape-Typen können hinzugefügt werden, ohne den bestehenden Code zu ändern, indem einfach eine neue Implementierung des Interfaces erstellt und zur Liste hinzugefügt wird. Jede Strategy-Implementierung ist dabei für einen spezifischen PropertyShape-Typ verantwortlich und kann unabhängig entwickelt und getestet werden. [67]

Die Einträge der o.g. Liste sind Implementierungen dieses Interfaces. Im Zeitraum bis zur Fertigstellung dieser Arbeit bestand die Liste aus Implementierungen zur Generierung der `-cardinality`, `-valueType`, `-hasType`, `-datatype` und `-inverseCardinality` PropertyShapes. Soll in Zukunft eine weitere PropertyShape erstellt werden, kann die Liste einfach um die Implementierung der Generierungsklasse erweitert werden.

Die Implementierungen der Interfaces überprüfen zuerst, ob der notwendige Kontext gesetzt ist und werfen gegebenenfalls eine `IllegalStateException`, um auf Implementierungsfehler hinzuweisen. Falls das PropertyShape nicht für die Art von Property, welche betrachtet wird, erstellt werden soll, wird null als Indikator für das Nicht-Erstellen eines PropertyShapes zurückgegeben. Es soll zum Beispiel kein `-inverseCardinality` PropertyShape für ein Attribut erstellt werden.

Um das Herauslesen der Informationen aus der Ontologie und das eigentliche Erstellen der Shapes weiter aufzuteilen, wurden Builder für die jeweiligen PropertyShapes erstellt. Diese bieten erneut ein Fluent-Interface mit einer abschließenden `build()`-Methode, welche die erstellte Ressource zurückgibt. Der Vorteil dieser Aufteilung ist, dass das eigentliche Arbeiten mit Triples fast komplett aus den Generator-Klassen wegabstrahiert werden kann und diese somit stark vereinfacht werden. Die Implementierung der `createPropertyShape(Resource property)` Methode der für `-inverseCardinality` PropertyShapes zuständigen Klasse ist als beispielhafter Auszug in Listing 32 abgebildet.

```
@Override
public Resource createPropertyShape(Resource association) {
    if (ontologyModel == null || shaclModel == null || shaclPrefix == null) {
        throw new IllegalStateException(…);
    }
    if (!CIMPropertyUtils.isAssociation(association)
        || !shouldCreateInverseCardinalityPropertyShape(association)) {
        return null;
    }
    var order = CIMPropertySHACLUtils.getOrder(ontologyModel,association.getURI());
    var multiplicity = CIMPropertyUtils.resolveMultiplicity(association);
    return new InverseCardinalityPropertyShapeBuilder(shaclModel)
        .setPrefixEntry(shaclPrefix)
        .setPropertyUri(association.getURI())
        .setInversePropertyUri(association.getProperty(CIMS.inverseRoleName)
            .getResource()
            .getURI())
        .setPropertyGroupUri(shaclPrefix.getUri() + PROPERTY_GROUP_LABEL)
        .setOrder(order)
        .setLowerBound(multiplicity.lowerBound())
        .setUpperBound(multiplicity.upperBound())
        .build();
}
```

Listing 32: createPropertyShape Methode für inverseCardinality PropertyShapes

Um ein unnötiges doppeltes Erstellen der PropertyShapes zu vermeiden, werden die generierten PropertyShapes als Set in der SHACLFromCIMGenerator-Klasse in einer Map mit der zugehörigen Property als Key gespeichert. Erbt eine Klasse eine Property, für welche schon PropertyShapes erstellt wurden, können diese aus der Map ausgelesen werden. Diese einfache Caching Strategie eliminiert viele sich wiederholende Generierungsabläufe und verbessert die Laufzeit der Generierung.

Generierung NodeShapes

Nachdem für jede Property der Klasse die zugehörigen PropertyShapes erstellt wurden, wird anschließend das NodeShape für die Klasse erstellt. Dieses erfolgt erneut über eine Klasse `NodeShapeBuilder`, welche den Gebrauch von Triples wegabstrahiert und das NodeShape als Ressource zurückgibt. Da pro Klasse nur ein NodeShape erstellt wird und, wie in Listing 33 zu sehen, sehr simpel und kurz ist, wird die Nutzung des Builders nicht in eine separate Klasse, sondern nur in eine interne private Methode ausgelagert.

```
private void createNodeShape(Resource classResource, Set<Resource> propertyShapes){
    var nodeShape = new NodeShapeBuilder(shacl)
        .setClosed(closed)
        .setPrefixEntry(shaclPrefix)
        .setTargetClassUri(classResource.getURI())
        .setPropertyShapes(propertyShapes)
        .build();
    shacl.add(nodeShape.listProperties());
}
```

Listing 33: createNodeShape Methode

PropertyGroups

Nachdem alle Klassen und Properties betrachtet wurden, werden zuletzt die PropertyGroups erstellt. Deren Erstellung erfolgt dadurch, dass aus dem Zwischenstand des SHACL-Models alle Objekte des `sh:propertyGroup` Statements der PropertyShapes ausgelesen werden. Für jede gefundene IRI wird anschließend eine PropertyGroup erstellt. Der Vorteil dieses Ansatzes gegenüber einer manuellen Definition und Erstellung der PropertyGroups ist, dass nicht darauf geachtet werden muss, dass PropertyShapes eine an anderer Stelle definierte PropertyGroup referenzieren. Außerdem gibt es so keine ungenutzten PropertyGroups, die das SHACL-Model ohne Mehrwert füllen würden.

5.2.2 Implementierung des XSD Datatype-Mappings

Im Falle, dass es sich bei einer CIM-Property um ein Attribut handelt, muss dessen Datentyp ermittelt werden. In CIM werden, anstatt direkt XSD-Datentypen anzugeben, Klassen verwendet, welche in einen Primitiven Typen überführt werden können. Die Methode `public static RDFDatatype classLabelToDatatype(String primitiveDatatypeLabel)` der Utility-Klasse `XSDDatatypeMapper` erhält das Label der Klasse als Argument und konstruiert daraus zunächst eine potentielle XSD-IRI durch Voranstellung des XSD-Präfixes. Anschließend wird diese IRI case-insensitive gegen die Menge der standardisierten XSD-Datentypen validiert. Falls es eine Übereinstimmung gibt, wird dieser XSD-Datentyp zurückgegeben. Falls eine Zuordnung nicht möglich ist, wird die erstellte IRI als Datentyp zurückgegeben. Dieser wird von der SHACL-Validierungsengine nicht verwertet werden können und dabei einen Fehler werfen, welcher den Nutzer schnell auf diesen aufmerksam macht und er diesen relativ einfach händisch ausbessern kann. Zwar könnte an dieser Stelle auch direkt eine Exception geworfen werden, welche die Generierung abbricht, jedoch wird dies nicht getan um nicht zwingend notwendige Unterbrechungen zu vermeiden.

5.2.3 Tests und Validierung der generierten SHACL-Regeln

Um sicherzustellen, dass die generierten SHACL-Regeln valide sind, wurden mehrere Tests implementiert. Hierbei wurden nicht die einzelnen Klassen der Generierungs-Logik getestet, sondern der gesamte generierte SHACL-Graph als End-to-End-Validierung überprüft.

Die Tests nutzen die in 8.2.1 manuell definierte CIM-Ontologie, um zugehörige, definierte Instanzdaten mittels des Apache Jena SHACL-Validators zu validieren. Diese Instanzdaten decken systematisch die erstellten Constraints ab, wobei es immer mindestens einen Instanzgraphen für den Positiv- und einen für den Negativ-Fall eines PropertyShapes existieren. Für Kardinalitäts-PropertyShapes werden beispielsweise Fälle für das Fehlen eines required Attributes, das zu häufige Vorkommen einer Assoziation und auch der Positivfall getestet.

Alle Tests werden sowohl für open als auch für closed Shapes getestet, wobei dies bei fast allen Tests diese Änderungen keinen Unterschied ergibt. Lediglich für diesen Fall spezifische Tests schlagen hier an.

Die `SHACLFromCIMGeneratorTest`-Klasse ist aufgeteilt in mehrere Unterklassen zur besseren Strukturierung: `FullyValidGraphTests`, welche Positivfälle beinhaltet,

`CardinalityTests`, welche Kardinalitäts-spezifische Tests enthält, `DatatypeTests`, die Attributdatentypen validiert, `ValueTypeTests`, die die `-hasType` und `-valueType` PropertyShapes, die Assoziationsdatentypen validiert, `ClosedShapeTests`, welche die o.g. Tests für closed und open Shapes beinhaltet und zuletzt die `MixedValidationTests`, welche Graphen testet, die sowohl teilweise valide als auch invalide sind.

Alle Tests sind wie im RDF-Architect für eine klare Teststruktur nach dem Arrange-Act-assert Pattern geschrieben und folgen der Namenskonvention "methode_testFall_erwartetesErgebnis". Die mit Assertj geschriebenen Assertions validieren detailliert, die Eigenschaften des SHACL-Report-Entries, welche im Code definiert sind.

Listing 34 ist ein beispielhafter Test, welcher die in Anhang 8.2.2 definierte Testdatei validiert und auf die erwartete Warnung für die Ressource `http://example.org/instances#child` prüft.

```
@ParameterizedTest
@ValueSource(booleans = {true, false})
void validate_MissingReferencedRessource_warns(Boolean closed) {
    // Arrange
    var ontology = readFile("ontology.ttl");
    var instance = readFile("hasTypeWarningMissingReferencedClass.ttl.ttl");

    // Act
    var shacl = new SHACLFromCIMGenerator(ontology,
                                          PREFIX_ENTRY,
                                          closed).generate();
    var report = ShaclValidator.get().validate(shacl.getGraph(),
                                               instance.getGraph());

    // Assert
    assertThat(report.conforms()).isFalse();
    assertThat(report.getEntries()).hasSize(1);
    var entry1 = report.getEntries().iterator().next();
    assertThat(entry1.severity()).isEqualTo(Severity.Warning);
     assertThat(entry1.focusNode()
                      .getURI())
                      .isEqualTo("http://example.org/instances#child");
    assertThat(entry1.resultPath()).hasToString(…);
    assertThat(entry1.message()).isEqualTo(…);
}
```

Listing 34: Test MissingReferencedRessource

5.3 Umsetzung der Visualisierung

Die Visualisierung der SHACL-Regeln wurde vollständig in das bestehende SvelteKit Frontend des RDF-Architects integriert. Svelte 5 ermöglicht durch sein reaktives Komponentensystem eine modulare und wartbare Architektur, bei der jede SHACL-Ansicht eine eigenständige Komponente ist. Diese Komponenten kapseln sowohl die Darstellungslogik als auch die Datenbeschaffung vollständig, indem sie intern über die native Fetch-API mit den Backend-REST-Endpunkten kommunizieren. Dadurch entsteht eine lose Kopplung zwischen den UI-

Komponenten und dem Rest der Anwendung, was unnötig komplexe Abhängigkeiten zwischen Komponenten minimiert.

Die Komponenten verwalten ihren eigenen Zustand über Sveltes reaktive Variablen und lösen ein automatisches Re-Rendering aus, wenn Aktionen getätigt werden, welche die zugrundeliegenden SHACL-Daten ändern könnten. Diese Architektur folgt dem Single Responsibility Principle, da jede Komponente ausschließlich für einen spezifischen Aspekt der SHACL-Erweiterung verantwortlich ist.

5.3.1 Realisierung der Grafischen Darstellung

Die Visualisierung der einzelnen Turtle-Codeabschnitte wurde über CodeMirror 6 mit dem zusätzlichen Package für die Turtle-Sprache realisiert. Um diese in Svelte einzubauen und ein einheitliches Aussehen des Editors zu gewährleisten sowie die Nutzung der Library zu vereinfachen, wurde eine Wrapper-Komponente entwickelt, welche CodeMirror kapselt. Falls in Zukunft gewünscht ist, eine andere Library zu nutzen, kann diese problemlos ausgetauscht werden.

```
<script>
    import { EditorView, basicSetup } from "codemirror";
    import { EditorState } from "@codemirror/state";
    import { turtle } from "codemirror-lang-turtle";
    import { onDestroy, onMount } from "svelte";

    let { value = $bindable(), readOnly = false } = $props();
    let editorDiv;
    let editor;
    onMount(() => {
        const state = EditorState.create({
            doc: value, // Set initial value
            extensions: [
                basicSetup,
                turtle(),
                //setup if the editor is readOnly
                EditorView.editable.of(!readOnly),
                //update value on change
                EditorView.updateListener.of(update => {
                    if (update.docChanged) {
                        value = update.state.doc.toString();
                    }
                }),
            ],
        });
        // Create CodeMirror editor instance
        editor = new EditorView({
            state,
            parent: editorDiv,
        });
    });
    onDestroy(() => {
        editor?.destroy();
    });
</script>
<div bind:this={editorDiv} class="w-full"></div>
```

Listing 35: TtlCodeEditor.svelte

Die Komponente ist in Listing 35 dargestellt. Sie bietet nach außen eine bindable Variable für den Turtle-String und optional die Möglichkeit zu definieren, ob der Code-Editor schreibgeschützt sein soll, wobei dieser Wert standardmäßig auf `false` steht. Die Nutzung der Komponente reduziert somit sich letztendlich auf:
`<TtlCodeEditor bind:value={TtlString} readOnly={true} />`.

5.3.2 Backend

Das Backend stellt über entsprechende API-Routen sowohl die custom als auch die generierten SHACL-Regeln und stellt gegebenenfalls die Möglichkeit für deren Bearbeitung in bestimmten Kontexten.

Wie in Kapitel 4.4 beschrieben, wird über die REST-basierte API immer auf einen bestimmen Graphen in einem bestimmten Dataset über `/api/datasets/{datasetName}/graphs/{graphURI}` eingeschränkt. Aus zeitlichen Gründen wurde jedoch nicht die Funktionalität implementiert, dass custom SHACL-Dateien Graph-spezifisch abgespeichert werden können. Derzeit besteht nur die Möglichkeit, eine einzelne custom SHACL-Datei für alle Graphen hochzuladen. Die API-Routen wurden trotzdem bereits unter der Annahme erstellt, dass diese Funktionalität existiert, wodurch für die endgültige Implementierung keine Frontend-Änderungen geschehen müssten und nur das Backend angepasst werden müsste.

Im Folgenden wird beschrieben wie diese Einteilung in die jeweiligen Kontexte und die Kommunikation mit dem Frontend implementiert ist.

Propertykontext

Die Daten für die Propertyansicht sind über die Route `GET …/classes/{classUUID}/attributes/{propertyUUID}/shacl` abfragbar. Analog dazu existiert für Assoziationen die Route `GET …/classes/{classUUID}/associations/{propertyUUID}/shacl`. Diese geben eine JSON-Struktur zurück, welche, wie in Listing 36 dargestellt, custom und generated PropertyShapes zurückgibt.

```
{
  "custom": {
    "prefixes": "<<TtlString>>",
    "propertyShapes": [
      {
      "id": "<<IRIString>>",
      "triples": "<<TtlString>>",
      "order":<<Integer>>}
    ],
  },
  "generated": { /* gleiche Struktur */ }
}
```

Listing 36: Returnformat PropertyShapes spezifisch für Property

Diese Struktur wird im Backend aufgebaut, nachdem die Shapes für den Kontext der Property generiert wurden. Daraufhin wird sowohl für die generierten als auch die importierten Shapes die Methode `public List<PropertyShape> getPropertyShapesOfProperty(Model ontology, String propertyUri)` der Klasse `SHACLShapesFetcher` aufgerufen, welche bei der Initialisierung das jeweilige SHACL-Model als Argument erhält. Diese Methode gibt eine Liste von PropertyShapes aus dem SHACL-Model zurück, die der Property zugeordnet werden können. Die Zuordnung selbst geschieht über die in Listing 37 beschriebene SPARQL-Query. In dieser werden die bekannten Arten, über welche eine Property von einer PropertyShape referenziert werden können, abgefragt.

```
PREFIX  rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX  sh:   <http://www.w3.org/ns/shacl#>
PREFIX  cims: <http://iec.ch/TC57/1999/rdf-schema-extensions-19990926#>

SELECT DISTINCT ?propertyShape
WHERE {
    {
        ?propertyShape  sh:path <PROPERTY_URI>.
    }
    UNION
    {
        ?propertyShape  sh:path ?list.
        ?list (rdf:rest*/ rdf:first) <PROPERTY_URI>.
    }
    UNION
    {
        ?propertyShape  sh:path [ sh:inversePath ?inverseRoleName  ].
        ?inverseRoleName cims:inverseRoleName <PROPERTY_URI> .
    }
    ?propertyShape  rdf:type  sh:PropertyShape.
}
```

Listing 37: Query Zuweisung PropertyShape zu Property

Die Ergebnisse dieser Query enthalten die IRIs der PropertyShapes. Diese werden anschließend zu TTL-Strings serialisiert und dem PropertyShape-Objekt angefügt, das der später zurückzugebenden Liste hinzugefügt wird. An dieses Objekt werden zusätzlich die Order der Shapes sowie ihre IRI als `id` angehängt. Die finale Liste wird nach dem `order`-Wert sortiert zurückgegeben. Diese werden dann zu der in Listing 36 beschriebenen Struktur serialisiert und an das Frontend zurückgegeben.

Klassenkontext

Die Daten für die Klassenansicht sind über die Route `GET …/classes/{classUUID}/shacl` abfragbar und liefern eine umfangreichere JSON-Struktur zurück, welche alle SHACL-Shapes im Kontext einer Klasse enthält. Diese in Listing 38 abgebildete Struktur umfasst neben den PropertyShapes auch die zugehörigen NodeShapes und unterscheidet zwischen direkten und geerbten Properties.

```
{
  "custom": {
    "prefixes": "<<String>>",
    "nodeShapes": [
      {
        "id": "<<String>>",
        "triples": "<<String>>",
      }
    ],
    "propertyShapes": [
      {
        "domain": "<<UUID>>",
        "label": "<<String>>",
        "propertyType": "<<String>>",
        "propertyShapes": [<<TtlString>>]
      }
    ],
    "derivedPropertyShapes": [/* gleiche Struktur wie propertyShapes */]
  },
  "generated": { /* gleiche Struktur */ }
}
```

Listing 38: Returnformat Klassenspezifische Ansicht

Diese Struktur wird analog zu der für den Property-spezifischen Kontext erstellt, nur dass hier für die custom und generierten SHACL-Regeln eine Instanz der `SHACLToClassRelations` erstellt wird. Diese beinhaltet die Präfixe der Klassen, eine Liste der NodeShapes, eine der PropertyShapes und eine der geerbten PropertyShapes. Das Zuordnen der Shapes zu der Klasse geschieht über die Klasse `PropertyShapeToClassAssigner`.

Die `PropertyShapeToClassAssigner` Klasse verwendet verschiedene SPARQL-Queries, um PropertyShapes systematisch nach ihrem Typ zu kategorisieren. Zunächst werden Attribut-PropertyShapes über Properties mit `cims:stereotype attribute` identifiziert und deren Zuordnung über `sh:path` ermittelt. Anschließend erfolgt die Erfassung der Assoziations-PropertyShapes über Properties mit `cims:inverseRoleName`, wobei sowohl direkte `sh:path` Zuordnungen als auch inverse Pfade berücksichtigt werden. Als dritte Kategorie werden SPARQL-Constraints identifiziert, welche sich durch das Vorhandensein eines `sh:sparql` Prädikats auszeichnen. Abschließend werden alle verbleibenden PropertyShapes erfasst, die über `sh:property` den Nodeshapes der Klasse zugeordnet sind jedoch nicht in die vorherigen Kategorien fallen.

Eine besondere Behandlung erfahren geerbte Properties, die in der separaten `derivedPropertyShapes` Sektion aufgeführt werden und durch die `public List<PropertyShapesWrapper> getDerivedPropertyShapesOfClass(UUID classUUID)` Methode ermittelt werden. Über `public List<CIMClass> findSuperClasses(UUID classUUID)` wird die komplette Vererbungshierarchie traversiert und anschließend die PropertyShapes aller ermittelten Superklassen gesammelt. Die finale Liste wird sowohl alphabetisch als auch nach `sh:order` Werten sortiert, um eine konsistente Darstellung zu gewährleisten.

Gesamtkontext

Für die vollständige SHACL-Ansicht stellt das Backend die API-Routen `GET …/shacl/generate/string` und `GET …/shacl/custom/string` bereit, die direkt TTL-Strings ohne weitere JSON-Strukturierung zurückgeben.

Für generierte SHACL-Regeln wird die `generate()` Methode der `SHACLFromCIMGenerator` Klasse aufgerufen, welche das komplette SHACL-Model für alle instanziierbaren Klassen der Ontologie erstellt. Das resultierende Apache Jena Model wird anschließend vollständig zu einem TTL-String serialisiert. Custom SHACL-Regeln werden direkt aus dem importierten SHACL-Graphen ausgelesen und ebenfalls als TTL-String serialisiert.

Frontend-Implementierung der SHACL-Dialoge

Alle SHACL-Ansichten sind als Svelte-Komponenten implementiert, die als Modal-Dialoge über dem Hauptfenster angezeigt werden. Diese gemeinsame Architektur gewährleistet eine konsistente Benutzererfahrung und vereinfacht die Wartung des Codes.

Beim Öffnen eines Dialogs werden automatisch Fetch-Calls an die entsprechenden Backend-APIs ausgeführt, um sowohl die generierten als auch die custom SHACL-Regeln zu laden. Die Darstellung erfolgt über die zuvor beschriebene `TtlCodeEditor`-Komponente, welche Syntax-Highlighting und Editier-Funktionalitäten bereitstellt.

Alle Dialoge implementieren ein einheitliches Tab-System mit zwei Bereichen: "Generated SHACL" wird ausschließlich im Read-only-Modus angezeigt, während "Custom SHACL" vom Benutzer bearbeitet werden kann.

Change-Detection und Save-Button

Für die Erkennung von Änderungen wird beim Laden der Daten der ursprüngliche Wert der custom SHACL-Regeln in einer separaten Variable gespeichert. Bei jeder Texteingabe im `TtlCodeEditor` erfolgt ein Vergleich des aktuellen Inhalts mit dem ursprünglichen Wert. Sobald eine Abweichung festgestellt wird, wird über eine reaktive Svelte-Variable ein "Save changes" Button eingeblendet, der dem Benutzer das Speichern der Änderungen ermöglicht.

PUT-Request Implementierung

Das Speichern funktioniert nach dem Prinzip des kompletten Ersetzens des jeweiligen Kontexts. Das Frontend kennt dabei den gesamten SHACL-Kontext der entsprechenden Property, Klasse oder des ganzen Graphen und muss einen validen TTL-String aufbauen, der alle aktualisierten SHACL-Shapes für diesen Kontext enthält. Dieser wird über die jeweilige PUT-Route an das Backend gesendet, wobei für den Property-Kontext `PUT …/classes/{classUUID}/attributes/{attributeUUID}/shacl`, für den Klassen-Kontext `PUT …/classes/{classUUID}/shacl/custom` und für den vollständigen Kontext `PUT …/shacl/custom/string` verwendet wird.

Das Backend löscht dabei zunächst die bestehenden SHACL-Regeln für den spezifischen Kontext vollständig und ersetzt sie anschließend durch den im Request-Body übertragenen

TTL-String. Im Falle einer invaliden Eingabe wird die Request abgelehnt ohne Änderungen vorzunehmen.

Nach erfolgreichem Speichern wird die gesamte SHACL-Ansicht neu geladen, um Konsistenz zwischen Frontend und Backend zu gewährleisten.

5.3.3 Import/Export

Beim Export von kombinierten SHACL-Regeln wird eine spezielle Merge-Strategie verwendet, die über die `ModelResourceExclusiveMerge` Klasse implementiert ist. Diese gewährleistet, dass bei IRI-Konflikten zwischen custom und generierten SHACL-Regeln ausschließlich die custom Regeln übernommen werden.

```
@Override
public Model merge(Model primary, Model secondary) {
    var resultModel = ModelFactory.createModelForGraph(
                                GraphUtils.deepCopy(primary.getGraph()));
    var primaryPrefixes = primary.getNsPrefixMap();
    secondary.getNsPrefixMap()
        .entrySet()
        .stream()
        .filter(entry -> !primaryPrefixes.containsKey(entry.getKey()))
        .forEach(entry -> primaryPrefixes.put(entry.getKey(), entry.getValue()));
    resultModel.setNsPrefixes(primaryPrefixes);

    secondary.listStatements()
        .toSet()
        .stream()
        .filter(statement -> {
            return !primary.containsResource(
                        primary.createResource(statement.getSubject()
                                                .getURI())
            );
        })
        .filter(statement -> !statement.getSubject().isAnon())
            .forEach(statement -> copyStatementToModel(secondary,
                                                        resultModel,
                                                        statement));
    return resultModel;
}
```

Listing 39: merge-Methode das Zusammenführen von custom und merged SHACL

Der Merge-Prozess beginnt wie in Listing 39 beschrieben mit der vollständigen Kopie des primary Models über `GraphUtils.deepCopy(primary.getGraph())`, wodurch alle custom SHACL-Regeln als Basis übernommen werden. Anschließend erfolgt die Zusammenführung der Namespace-Präfixe beider Modelle.

Die eigentliche Merge-Logik arbeitet Statement-basiert: Über `secondary.listStatements().toSet().stream()` werden alle Statements des secondary Models, also die generierten SHACL Regeln durchlaufen. Der erste Filter `!primary.containsResource(primary.createResource(statement.getSubject().getURI()))` prüft, ob die Subject-Ressource bereits im primary Model

existiert. Ist dies der Fall, wird das gesamte Statement ignoriert. Diese Strategie verhindert, dass einzelne Triples aus verschiedenen Modellen für dieselbe Ressource gemischt werden, was zu inkonsistenten SHACL-Shapes führen könnte. Der zweite Filter `!statement.getSubject().isAnon()` schließt anonyme Ressourcen aus der direkten Verarbeitung aus, da diese schon an anderer Stelle behandelt werden. Auf diese Weise werden Blank Nodes nur kopiert, wenn es eine Referenz auf diese gibt. Damit wird verhindert, dass verwaiste Blank Nodes in das Ergebnismodel eingefügt werden. Diese Statements mit Blank Nodes als Objekt erfordern, dass das Kopieren von Statements wie in Listing 40 rekursiv beschrieben wird, um verschachtelte SHACL-Konstrukte korrekt zu kopieren.

```
private void copyStatementToModel(Model originalModel,
                                  Model newModel,
                                  Statement statement)
{
    newModel.add(statement);
    var object = statement.getObject();
    if (object.isAnon()) {
        originalModel.listStatements(object.asResource(),
                                     null,
                                     (RDFNode) null)
            .forEach(stmt -> copyStatementToModel(originalModel,
                                                  newModel,
                                                  stmt));
    }
}
```

Listing 40: copyStatementToModel-Methode für das korrekte Kopieren von Blank Nodes

Diese Methode fügt zunächst das Statement dem Ergebnismodel hinzu und ruft sich selbst rekursiv auf, falls das Statement auf eine Blank Node zeigt.

Die erfolgreiche Implementierung der automatischen SHACL-Generierung im RDF-Architect stellt einen wichtigen Schritt zur Lösung der anfangs beschriebenen Problemstellung dar. Durch die Integration der Validierungslogik direkt in den Editor werden Inkonsistenzen zwischen Modell und SHACL-Regeln vermieden und die Wartung der Validierungen automatisiert.

6 Fazit und Ausblick

6.1 Zielerreichung und Bewertung der Lösung

Im Rahmen der Arbeit konnte erfolgreich eine Erweiterung des RDF-Architects umgesetzt werden, welche die automatische Generierung und benutzerfreundliche Visualisierung von SHACL-Regeln für CIM/XML-Ontologien ermöglicht. Diese Lösung schließt die Lücke der manuellen und fehleranfälligen Erstellung von Validierungsregeln im Modellierungsprozess und integriert deren Wartung direkt in diesen.

Es konnten alle wesentlichen der in Kapitel 3 beschriebenen Muss-Anforderungen erfolgreich umgesetzt werden. Die implementierte Lösung generiert automatisch strukturell ableitbare SHACL-Shapes aus CIM/XML-Ontologien, bietet zusätzlich Import/Export- sowie Bearbeitungs-Funktionalität für custom SHACL-Regeln und ermöglicht kontextuelle Visualisierung auf Property-, Klassen- und Gesamtebene.

Einige untergeordnete Soll-Anforderungen konnten aufgrund zeitlicher Beschränkungen nicht vollständig umgesetzt werden, insbesondere die Graph-spezifische Speicherung von SHACL-Regeln, die eine grundlegende Erweiterung der RDF-Architect-Datenhaltung erfordert hätte. Diese Funktionalität ist jedoch in der zukünftigen Entwicklung des RDF-Architects vorgesehen.

Die entwickelte Lösung zeichnet sich durch mehrere besondere Merkmale aus: Die modulare und erweiterbare Architektur basiert auf dem Strategy Pattern und ermöglicht das einfache Hinzufügen neuer PropertyShape-Typen. Die Implementierung deckt alle strukturellen Validierungen für CIM-Klasseneigenschaften ab und behandelt die Vererbungshierarchien einschließlich zirkulärer Referenzen korrekt.

Die Lösung konnte bereits insofern bei SOPTIM intern erfolgreich eingesetzt werden, als dass die SHACL-Erweiterung benötigte Validierungsregeln für CIM/XML-Ontologien nach dem CGMES 2.4.15 generieren konnte und damit ihre praktische Anwendbarkeit unter Beweis gestellt hat.

Bezüglich der in Kapitel 3.2 formulierten Forschungsfragen konnte die Arbeit ebenfalls wichtige Erkenntnisse liefern. Zur ersten Forschungsfrage bezüglich der Vollständigkeit und Automatisierbarkeit der ENTSO-E SHACL-Shapes zeigt die systematische Analyse der für CGMES 3.0 veröffentlichten SHACL-Shapes, dass Graph-spezifische strukturelle Validierungen durch automatische Generierung abgedeckt werden können. Dies umfasst Kardinalitäts-, Datentyp-, Wertetyp- und inverse Kardinalitätsvalidierungen für alle Properties der CIM-Klassen. Nicht automatisch generierbar sind hingegen domänenspezifische SPARQL-Constraints für energiewirtschaftliche Zusammenhänge sowie Standard-Compliance-Regeln wie IEC-Namenskonventionen, da diese externes domänenspezifisches Wissen erfordern, das nicht aus der UML-Struktur ableitbar ist.

Zur zweiten Forschungsfrage bezüglich der benutzerfreundlichen Integration von SHACL-Funktionalitäten wurde eine dreistufige kontextuelle Visualisierung entwickelt, die Property-spezifische, klassenspezifische und Gesamtansichten bietet. Durch die intuitive Zuordnung der SHACL-Regeln zu den entsprechenden UML-Elementen und die Integration von Syntax-Highlighting im menschenlesbaren Turtle-Format können auch Benutzer ohne umfangreiche SHACL-Vorkenntnisse effektiv mit den Validierungsregeln arbeiten und diese verstehen.

6.2 Limitationen der aktuellen Implementierung

Aufgrund zeitlicher Beschränkungen ergab sich eine wesentliche Limitierung in der Implementierung. Es konnte die geplante Zuordnung von importierten SHACL-Regeln zu einzelnen Graphen nicht vollständig umgesetzt werden. Der Zeitaufwand, in die bestehende Architektur und Datenhaltung des RDF-Architects einzugreifen und dort Anpassungen vorzunehmen, wäre unverhältnismäßig dafür, dass diese Funktionalität ein nicht essentielles Ziel dieser Arbeit darstellt. Das konzeptionelle Design sieht vor, dass jeder Graph seine eigenen benutzerdefinierten SHACL-Regeln verwaltet und diese kontextspezifisch abgerufen werden können. Die aktuelle Implementierung unterstützt jedoch nur eine globale custom SHACL-Datei für den gesamten RDF-Architect-Kontext. Dies bedeutet, dass importierte SHACL-Regeln nicht Graph-spezifisch gespeichert und abgefragt werden können, was die beschriebenen kontextspezifischen API-Endpunkte einschränkt. Diese wurden in Hinblick darauf, dass diese Funktionalität in Zukunft hinzugefügt werden soll, trotzdem bereits entsprechend implementiert. Es wird lediglich im Backend die Auswahl eines bestimmten Graphen ignoriert.

Aus funktionaler Sicht bestehen außerdem Grenzen bei domänenspezifischen Einschränkungen. Es ist beispielsweise nicht möglich, Standard-Compliance-Regeln wie IEC-Namenskonventionen und ENTSO-E-spezifische Vorgaben aus den UML-Strukturen abzuleiten.

Es werden außerdem noch keine Multi-Profil-Szenarien unterstützt, da für diese zuerst der RDF-Architect selbst um diese Funktionalität erweitert werden müsste.

6.3 Ausblick und Weiterentwicklungsmöglichkeiten

Für die Weiterentwicklung stehen vor allem zwei Aufgaben im Vordergrund. Erstens wurde die Entwicklung des RDF-Architects während der Entwicklung der SHACL-Erweiterung fortgesetzt, wodurch einige Funktionalitäten der Erweiterung mittlerweile wieder Anpassungen benötigen. Es sollte also zuerst der aktuelle Stand der SHACL-Erweiterung an den aktuellen Entwicklungsstand des RDF-Architects angepasst werden. Zweitens sollte die o.g. Implementierung der Graph-spezifischen SHACL-Verwaltung durch eine 1:1-Zuordnung zwischen Ontologie- und SHACL-Graphen implementiert werden.

Weitere mögliche Entwicklungen umfassen die Anpassung an die geplante RDF-Architect-Funktionalität für zusammenhängende CGMES-Profile, um Multi-Profil-Unterstützung zu ermöglichen. Zusätzlich könnten erweiterte Konfigurierbarkeitsoptionen wie Benutzereinstellungen für closed/open NodeShapes sowie dem Einstellen eines Benutzer-definierten Präfixes für die generierten SHACL-Regeln implementiert werden. Ein erweitertes Validierungs-UI für Live-Validierung von Instanzdaten würde die Benutzerfreundlichkeit weiter steigern. Die Unterstützung für SPARQL-Constraints durch benutzerfreundliche Erstellung komplexer Validierungsregeln wäre ebenfalls eine wertvolle Ergänzung.

Die bereits stattgefundene erfolgreiche interne Nutzung bei SOPTIM zeigt das Potenzial der Lösung auf. Im Rahmen der allgemein für RDF-Architect geplanten Open-Source-Strategie ist auch eine Veröffentlichung der SHACL-Erweiterung zu erwarten, was weitere Entwicklungsimpulse und Anwendungsszenarien ermöglichen könnte.

7 Quellenverzeichnis

[1] https://www.iwh-halle.de/fileadmin/user_upload/publications/wirtschaft_im_wandel/12-05-5.pdf. [Zugriff am 7 August 2025].

[2] https://www.entsoe.eu/about/inside-entsoe/objectives/. [Zugriff am 7 August 2025].

[3] https://www.entsoe.eu/digital/common-information-model/. [Zugriff am 7 August 2025].

[4] https://ieeexplore.ieee.org/document/8249433. [Zugriff am 7 August 2025].

[5] https://www.soptim.de/en/company/. [Zugriff am 7 August 2025].

[6] https://www.w3.org/TR/rdf11-concepts/. [Zugriff am 31 Juli 2025].

[7] https://www.w3.org/TR/rdf11-concepts/#resources-and-statements. [Zugriff am 3 August 2025].

[8] https://www.w3.org/TR/rdf11-concepts/#section-IRIs. [Zugriff am 31 Juli 2025].

[9] https://www.ietf.org/rfc/rfc3987.txt. [Zugriff am 31 Juli 2025].

[10] https://www.w3.org/TR/rdf-syntax-grammar/#section-Namespace. [Zugriff am 8 August 2025].

[11] https://www.w3.org/TR/rdf11-concepts/#section-triples. [Zugriff am 3 August 2025].

[12] https://www.w3.org/TR/rdf11-concepts/#section-blank-nodes. [Zugriff am 3 August 2025].

[13] https://www.w3.org/TR/rdf11-concepts/#data-model. [Zugriff am 3 August 2025].

[14] https://www.w3.org/TR/turtle/. [Zugriff am 31 Juli 2025].

[15] https://www.w3.org/TR/rdf-syntax-grammar/. [Zugriff am 31 Juli 2025].

[16] https://www.w3.org/TR/sparql11-query/. [Zugriff am 31 Juli 2025].

[17] https://www.w3.org/TR/rdf-schema/. [Zugriff am 4 August 2025].

[18] https://www.uml.org/what-is-uml.htm. [Zugriff am 6 August 2025].

[19] https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/. [Zugriff am 6 August 2025].

[20] https://tomgruber.org/writing/definition-of-ontology.pdf. [Zugriff am 7 August 2025].

[21] https://www.dmtf.org/standards/cim. [Zugriff am 7 August 2025].

[22] https://www.iec.ch/blog/iec-common-information-model-under-spotlight-1. [Zugriff am 7 August 2025].

[23] https://eepublicdownloads.entsoe.eu/clean-documents/CIM_documents/IOP/CGMES_2_5_TechnicalSpecification_61970-600_Part%201_Ed2.pdf. [Zugriff am 7 August 2025].

[24] https://www.ontotext.com/knowledgehub/fundamentals/what-is-shacl/. [Zugriff am 7 August 2025].

[25] https://www.w3.org/TR/shacl/#core-components-count. [Zugriff am 7 August 2025].

[26] https://www.w3.org/TR/shacl/#core-components. [Zugriff am 7 August 2025].

[27] https://www.w3.org/TR/shacl/. [Zugriff am 6 August 2025].

[28] https://www.w3.org/TR/shacl/#validation. [Zugriff am 7 August 2025].

[29] https://www.w3.org/TR/shacl/#constraints-section. [Zugriff am 7 August 2025].

[30] https://spring.io/projects/spring-framework. [Zugriff am 7 August 2025].

[31] https://github.com/spring-projects/spring-framework. [Zugriff am 7 August 2025].

[32] https://spring.io/projects/spring-boot. [Zugriff am 7 August 2025].

[33] https://github.com/spring-projects/spring-boot. [Zugriff am 7 August 2025].

[34] https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-requestmapping.html. [Zugriff am 7 August 2025].

[35] https://jena.apache.org/. [Zugriff am 7 August 2025].

[36] https://github.com/apache/jena. [Zugriff am 7 August 2025].

[37] https://jena.apache.org/documentation/rdf/index.html. [Zugriff am 7 August 2025].

[38] https://jena.apache.org/documentation/query/app_api.html. [Zugriff am 7 August 2025].

[39] https://jena.apache.org/documentation/shacl/. [Zugriff am 7 August 2025].

[40] https://junit.org/. [Zugriff am 7 August 2025].

[41] https://github.com/junit-team/junit-framework/. [Zugriff am 7 August 2025].

[42] https://joel-costigliola.github.io/assertj/. [Zugriff am 7 August 2025].

[43] https://github.com/assertj/assertj. [Zugriff am 7 August 2025].

[44] https://docs.junit.org/current/user-guide/. [Zugriff am 7 August 2025].

[45] https://assertj.github.io/doc/. [Zugriff am 7 August 2025].

[46] https://projectlombok.org/. [Zugriff am 7 August 2025].

[47] https://github.com/projectlombok/lombok. [Zugriff am 7 August 2025].

[48] https://projectlombok.org/features/Data. [Zugriff am 7 August 2025].

[49] https://projectlombok.org/features/Builder. [Zugriff am 7 August 2025].

[50] https://github.com/sveltejs/kit. [Zugriff am 7 August 2025].

[51] https://svelte.dev/docs/kit/introduction. [Zugriff am 7 Agust 2025].

[52] https://svelte.dev/. [Zugriff am 7 August 2025].

[53] https://github.com/sveltejs/svelte. [Zugriff am 7 August 2025].

[54] https://svelte.dev/blog/virtual-dom-is-pure-overhead. [Zugriff am 7 August 2025].

[55] https://svelte.dev/docs/svelte/what-are-runes. [Zugriff am 7 August 2025].

[56] https://tailwindcss.com/. [Zugriff am 7 August 2025].

[57] https://github.com/tailwindlabs/tailwindcss). [Zugriff am 7 August 2025].

[58] https://v3.tailwindcss.com/docs/utility-first. [Zugriff am 7 August 2025].

[59] https://prismjs.com/. [Zugriff am 7 August 2025].

[60] https://github.com/PrismJS/prism. [Zugriff am 7 August 2025].

[61] https://prismjs.com/#full-list-of-features. [Zugriff am 7 August 2025].

[62] https://codemirror.net/. [Zugriff am 7 August 2025].

[63] https://github.com/codemirror/dev/. [Zugriff am 7 August 2025].

[64] https://github.com/aatauil/codemirror-lang-turtle. [Zugriff am 7 August 2025].

[65] https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. [Zugriff am 7 August 2025].

[66] https://www.ibm.com/think/topics/rest-apis. [Zugriff am 7 August 2025].

[67] https://refactoring.guru/design-patterns/strategy. [Zugriff am 5 August 2025].

[68] https://www.w3.org/TR/sparql11-overview/. [Zugriff am 31 Juli 2025].

[69] https://www.omg.org/spec/UML/2.5/PDF. [Zugriff am 6 August 2025].

8 Anhang

8.1 Queries

8.1.1 Auflistung und Quantifizierung der SHACL-Ressourcentypen

Diese SPARQL-Query zählt, wie viele unterschiedliche IRI-Ressourcen in einem benannten Graphen jeweils einem bestimmten `rdf:type` zugeordnet sind, und liefert zusätzlich die Gesamtanzahl aller unterschiedlichen IRI-Subjekte im Graphen.

Angewendet auf das "DiagramLayoutProfile" des CGMES 3.0.2 wurden damit 69 Ressourcen identifiziert, die sich wie folgt aufteilen: 11 NodeShapes, 52 PropertyShapes, 5 PropertyGroups sowie eine `owl:Ontologie`.

```
SELECT ?type (COUNT(DISTINCT ?s) AS ?count)
WHERE {
    {
        GRAPH <http://example.com/shacl> {
            ?s a ?type .
            FILTER(isIRI(?s))
        }
    }
    UNION
    {
        GRAPH <http://example.com/shacl> {
            ?s ?p ?o .
            FILTER(isIRI(?s))
            BIND("ALL_IRI_SUBJECTS" AS ?type)
        }
    }
}
GROUP BY ?type
ORDER BY DESC(?count)
```

8.1.2 Quantifizierung und Zusammenhang von NodeShapes und Klassen

Diese SPARQL-Query erfasst die Anzahl aller definierten Klassen in der Ontologie, die Anzahl der SHACL-NodeShapes sowie wie vielen Klassen einem SHACL-NodeShape zugeordnet wird.

Angewendet auf das "DiagramLayoutProfile" des CGMES 3.0.2 wurden damit 18 Klassen ermittelt, für die 11 NodeShapes existieren, die 8 der 18 CIM-Klassen zugeordnet sind.

```
PREFIX rdf:     <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs:    <http://www.w3.org/2000/01/rdf-schema#>
PREFIX sh:      <http://www.w3.org/ns/shacl#>

SELECT
    (COUNT(DISTINCT ?class) AS ?classCount)
    (COUNT(DISTINCT ?NodeShape) AS ?NodeShapeCount)
    (COUNT(DISTINCT ?targetClass) AS ?ClassesReferencedByNodeShapes)
WHERE {
    {
        GRAPH <http://example.com/ontology> {
            ?class rdf:type rdfs:Class
        }
    }
```

```
    UNION
    { #NodeShapes
        GRAPH <http://example.com/shacl> {
            ?NodeShape rdf:type sh:NodeShape.
        }
    }
    UNION
    { #ClassesReferencedByPropertyShapes
        GRAPH <http://example.com/shacl> {
            ?AnyNodeShape
                rdf:type sh:NodeShape;
                sh:targetClass ?targetClass.
        }
        GRAPH <http://example.com/ontology> {
            ?targetClass rdf:type rdfs:Class
        }
    }
}
```

8.1.3 Relation von PropertyShapetypen zu Properties

Diese SPARQL-Query fragt ab, welche der SHACL-PropertyShape-Suffixe `-cardinality`, `-datatype` und `-valueType` einem Attribut und/oder einer Assoziation zugeordnet werden können.

Angewendet auf das "DiagramLayoutProfile" des CGMES 3.0.2 wurde damit ermittelt, dass `-cardinality` und `-datatype` einem Attribut und `-cardinality` und `-valueType` einer Assoziation zugehörig sind.

```
PREFIX cims:    <http://iec.ch/TC57/1999/rdf-schema-extensions-19990926#>
PREFIX rdf:     <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX sh:      <http://www.w3.org/ns/shacl#>
PREFIX nstd:    <http://iec.ch/TC57/NonStandard/UML#>

SELECT DISTINCT ?shapeType ?type
WHERE {
    {
        GRAPH <http://example.com/shacl> {
            ?propertyShape rdf:type sh:PropertyShape.
        {
        ?propertyShape sh:path ?property.
        }
        UNION
        {
            ?propertyShape sh:path (?property rdf:type).
        }
    }
    GRAPH <http://example.com/ontology> {
        ?property cims:stereotype nstd:attribute.
    }
    BIND("attribute" AS ?type)
    }
    UNION
    {
        GRAPH <http://example.com/shacl> {
            ?propertyShape rdf:type sh:PropertyShape.
        {
        ?propertyShape sh:path ?property.
```

```
        }
        UNION
        {
        ?propertyShape sh:path (?property rdf:type).
        }
    }
    GRAPH <http://example.com/ontology> {
        ?property cims:AssociationUsed ?any.
    }
    BIND("association" AS ?type)
    }
    # Extract the suffix from the URI
    BIND(
        IF(CONTAINS(STR(?propertyShape), "-cardinality"), "-cardinality",
            IF(CONTAINS(STR(?propertyShape), "-datatype"), "-datatype",
                IF(CONTAINS(STR(?propertyShape), "-valueType"), "-valueType", "unknown")
            )
        ) AS ?shapeType
    )
}
```

8.2 Tests

8.2.1 Ontology.ttl

Die Ontology.ttl ist die Datei, die das CIM-Model beinhaltet, welches für die Tests der Generierung genutzt wurde.

```
PREFIX cim:     <http://iec.ch/TC57/CIM100#>
PREFIX cims:    <http://iec.ch/TC57/1999/rdf-schema-extensions-19990926#>
PREFIX rdf:     <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs:    <http://www.w3.org/2000/01/rdf-schema#>

cim:Child.childEnumAtt
    rdf:type                rdf:Property;
    rdfs:domain             cim:Child;
    rdfs:label              "childEnumAtt"@en;
    rdfs:range              cim:EnumClass;
    cims:multiplicity       cims:M:1..1;
    cims:stereotype        <http://iec.ch/TC57/NonStandard/UML#attribute> .

cim:Parent.parentAtt
    rdf:type                rdf:Property;
    rdfs:domain             cim:Parent;
    rdfs:label              "parentAtt"@en;
    cims:dataType           cim:Float;
    cims:multiplicity       cims:M:1..1;
    cims:stereotype        <http://iec.ch/TC57/NonStandard/UML#attribute> .

cim:EnumClass
    rdf:type                rdfs:Class;
    rdfs:label              "EnumClass"@en;
    cims:belongsToCategory  cim:Package_Profile1;
    cims:stereotype          <http://iec.ch/TC57/NonStandard/UML#enumeration> .

cim:Parent
    rdf:type                rdfs:Class;
    rdfs:label              "Parent"@en;
    cims:belongsToCategory  cim:Package_Profile1 .
```

```
cim:Float
    rdf:type                  rdfs:Class;
    rdfs:label                "Float"@en;
    cims:belongsToCategory    cim:Package_Profile1;
    cims:stereotype           "Primitive" .

cim:EnumClass.entry3
    rdf:type                  cim:EnumClass;
    rdfs:label                "entry3"@en .

cim:EnumClass.entry1
    rdf:type                  cim:EnumClass;
    rdfs:label                "entry1"@en .

cim:Integer
    rdf:type                  rdfs:Class;
    rdfs:comment            "An integer number. The range is unspecified and not limited.";
    rdfs:label                "Integer"@en;
    cims:belongsToCategory    cim:Package_Profile1;
    cims:stereotype           "Primitive" .

cim:OtherClass
    rdf:type                  rdfs:Class;
    rdfs:label                "OtherClass"@en;
    cims:belongsToCategory    cim:Package_Profile1;
    cims:stereotype           <http://iec.ch/TC57/NonStandard/UML#concrete> .

cim:Child.childAtt
    rdf:type                  rdf:Property;
    rdfs:domain               cim:Child;
    rdfs:label                "childAtt"@en;
    cims:dataType             cim:Integer;
    cims:multiplicity         cims:M:0..1;
    cims:stereotype          <http://iec.ch/TC57/NonStandard/UML#attribute> .

cim:Child.OtherClass
    rdf:type                 rdf:Property;
    rdfs:domain              cim:Child;
    rdfs:label               "Child"@en;
    rdfs:range               cim:OtherClass;
    cims:AssociationUsed     "Yes";
    cims:inverseRoleName     cim:OtherClass.Child;
    cims:multiplicity        cims:M:0..1 .

cim:Child
    rdf:type                 rdfs:Class;
    rdfs:label               "Child"@en;
    rdfs:subClassOf          cim:Parent;
    cims:belongsToCategory cim:Package_Profile1;
    cims:stereotype          <http://iec.ch/TC57/NonStandard/UML#concrete> .

cim:OtherClass.Child
    rdf:type                 rdf:Property;
    rdfs:domain              cim:OtherClass;
    rdfs:label               "Child"@en;
    rdfs:range               cim:Child;
    cims:AssociationUsed     "No";
    cims:inverseRoleName     cim:Child.OtherClass;
    cims:multiplicity        cims:M:2..3 .

cim:EnumClass.entry2
    rdf:type                 cim:EnumClass;
```

```
    rdfs:label              "entry2"@en .

cim:Package_Profile1
    rdf:type                cims:ClassCategory;
    rdfs:label              "Profile1"@en .
```

8.2.2 hasTypeWarningMissingReferencedClass.ttl

Die Datei "hasTypeWarningMissingReferencedClass.ttl" enthält einen Instanzgraphen, bei dem eine Assoziation auf eine Ressource zeigt, die nicht existiert. Das Validieren dieses Graphen gegen die generierten SHACL-Regeln aus der Ontologie aus 8.2.1 wirft eine Warning, dass das `rdf:type` Triple der referenzierten Ressource nicht existiert und deshalb keine Typ-validierung ausgeführt werden kann.

```
PREFIX ex:      <http://example.org/instances#>
PREFIX cim:     <http://iec.ch/TC57/CIM100#>
PREFIX xsd:     <http://www.w3.org/2001/XMLSchema#>

ex:child
    rdf:type                cim:Child ;
    cim:Child.childAtt      42 ;
    cim:Child.childEnumAtt  cim:EnumClass.entry2 ;
    cim:Child.OtherClass    ex:other ;
    cim:Parent.parentAtt    "3.14"^^xsd:float .
```